설계보고서 제출서약서

나는 숭실대학교 소프트웨어학부의 일원으로 명예를 지키면서 생활하고 있습니다.
나는 보고서를 작성하면서 다음과 같은 사항을 준수하였음을 엄숙히 서약합니다.

1. 나는 자력으로 보고서를 작성하였습니다.
2. 나는 보고서에서 참조한 문헌의 출처를 밝혔으며 표절하지 않았습니다.
3. 나는 보고서의 내용을 조작하거나 날조하지 않았습니다.

교 과 목	캡스톤디자인
프로젝트 명	AI 투자자 기반 가상 주식시장 모의투자 대회 플랫폼
교과목 교수	이상준 교수님
지 도 교 수	정진홍 교수님
제 출 인	소프트웨어학부 3 학년 학번: (공개본 생략) 성명: 김영재 소프트웨어학부 3 학년 학번: (공개본 생략) 성명: 김경주 소프트웨어학부 3 학년 학번: (공개본 생략) 성명: 이경하 소프트웨어학부 3 학년 학번: (공개본 생략) 성명: 이윤성 소프트웨어학부 3 학년 학번: (공개본 생략) 성명: 박재호
제 출 일	2026년 11월 17일 (화)

AI 투자자 기반 가상 주식시장 모의투자 대회 플랫폼

캡스톤디자인 - 최종 설계제안서

제출일	2026.11.17	**소속학부**	소프트웨어학부
과목명	캡스톤디자인	**팀명**	hamburger
담당교수	이상준	**지도교수**	정진홍
팀원	김영재 김경주 이경하 이윤성 박재호		

- 목 차 -

1. 프로젝트 소개
2. 기존 시스템 분석
3. 특징 및 기능
4. 인터페이스 (UI)
5. 소프트웨어 요구사항 (Software Requirement)
6. 시스템 대블록 설계
7. 모듈 기반 소블록 인터페이스
8. 구성도 (Flow Chart)
9. 유즈케이스 다이어그램 (Usecase Diagram)
10. 클래스 다이어그램 (Class Diagram)
11. 시퀀스 다이어그램 (Sequence Diagram)
12. 알고리즘 구성 (Algorithm)
13. 데이터베이스 테이블 구조 (DB Table)
14. 시나리오 (Scenario)
15. 역할 분담 및 개발일정
16. 관련 기술 및 개념
17. 기대효과
18. 개발도구 및 개발환경
19. 참고문헌

1. 프로젝트 소개

본 프로젝트는 실제 시세를 가져오지 않고, 가상 기업으로 이루어진 자체 주식시장을 직접 설계·운영하는 모의투자 대회 플랫폼이다. 기존 모의투자 서비스는 실제 시세를 그대로 따라가기 때문에 사용자의 주문이 가격에 영향을 주지 못한다[1]. 본 플랫폼에서는 사용자의 주문이 자체 체결 엔진에서 실제로 체결되고, 개미·기관·외국인 등 서로 다른 성향의 AI 투자자(봇)가 함께 거래하면서 수요와 공급에 따라 가격이 형성된다. AI 투자자의 매매 성향은 외부 LLM이 시장 상황과 투자자 유형별 매매 통계[3]를 바탕으로 장중 2시간 단위로 산출하는 파라미터가 정한다.

시장에는 금리, 실적 발표, 자연재해, 정부 정책, 전쟁과 같은 사건이 발생하며, 봇들은 이를 각자의 성향에 따라 다르게 해석하여 매매한다. 사용자는 하나의 가상 주식장에서 투자 활동을 하고, 1개월·3개월·6개월·1년 단위의 기간별 대회에서 해당 기간의 투자 성과로 순위를 겨룬다. 가상 기업·가상 화폐·가상 사건만 사용하며, 실제 금융 상품의 거래나 투자 권유와는 무관하다.

※ 본문의 [숫자]는 19장 참고문헌의 번호이다.

2. 기존 시스템 분석

현재의 모의투자 서비스는 실제 시세에 맞춰 가상 자산을 사고파는 방식이다. 사용자의 주문은 가격 형성에 참여하지 않으므로 수요와 공급에 따라 가격이 움직이는 과정을 체험할 수 없고, 실제 시세에 종속되어 호황·불황·급변 같은 시장 상황을 임의로 구성하기도 어렵다. 또한 대회가 정해진 일정에만 열려 참여 시점이 제한된다. 따라서 사용자의 주문이 실제로 체결되어 가격이 형성되고, 다양한 시장 상황과 사건에 대해 서로 다른 투자자들이 반응하는 자체 시장이 필요하다. 이를 위해 아래의 서비스·자료·연구를 분석하였다.

2-1. 증권사 모의투자 서비스 [1]

구분	내용
개요	실제 시세에 맞춰 가상 자산을 사고파는 모의투자 서비스이다.
특징	실제 종목과 실제 시세를 사용하므로 실전 투자와 같은 화면과 주문 절차를 연습할 수 있다.
한계	사용자 주문이 가격에 영향을 주지 않는다. 시장 상황은 실제 시세에 종속되며, 대회는 정해진 일정에만 참여할 수 있다.
본 프로젝트와의 차이	플랫폼 내부 체결로 가격이 형성되며, 가상 기업과 가상 사건으로 시장 상황을 구성한다.

2-2. 한국거래소 투자자별 거래실적 [3]

구분	내용
개요	개인·기관·외국인 등 투자자 유형별 거래 통계를 공개하는 한국거래소 정보데이터시스템의 자료이다.
특징	시장 상황에 따라 투자자 유형별 매수·매도 경향이 어떻게 달라지는지 확인할 수 있다.
활용	페르소나 설계와 LLM 파라미터 산출의 참고 자료로 활용하여, 시장 상황에 따른 투자자 유형별 매매 경향을 실제 통계에 근거해 반영한다.

2-3. ABIDES (에이전트 기반 시장 시뮬레이터) [4]

구분	내용
개요	여러 유형의 거래 에이전트가 거래소 에이전트를 통해 거래하는 이산 사건 기반 시장 시뮬레이터이다.
특징	에이전트 유형별 행동 규칙과 거래소의 주문 처리를 분리하여, 에이전트 간 상호작용으로 가격이 형성되는 과정을 재현한다.
활용	투자자 유형별 행동 설계와 시장 사실성 검증 방법을 참고한다.

2-4. 한국거래소 매매 방식 [2]

구분	내용
개요	한국거래소 정규장의 매매 체결 규칙이다.
특징	정규장은 연속매매이며 가격 우선·시간 우선 원칙으로 체결하고, 지정가·시장가 등 주문 유형이 있다. 매매 시간은 09:00~15:30, 일일 가격 제한폭은 기준가격 대비 ±30%이다.
활용	C++ 체결 엔진의 매칭 규칙과 가상 주식장의 장 운영 시간·가격제한폭으로 구현한다.

2-5. 지정가 주문 장부(Limit Order Book) 연구 [5]

구분	내용
개요	주문 장부의 구조와 통계적 특성, 이를 설명하는 모형을 정리한 연구이다.
특징	가격 레벨별 대기 주문, 호가 간격, 주문 도착·취소 과정 등 주문 장부의 동작을 체계적으로 설명한다.
활용	호가창 자료구조와 체결 엔진 설계, 시장 검증 지표의 근거로 활용한다.

2-6. 기존 서비스의 한계점 및 보완 방안

한계점	보완 방안
가격 형성 과정 체험 불가	자체 체결 엔진에서 사용자와 봇의 주문이 함께 체결되어 가격이 형성되도록 한다.
시장 상황 연출의 한계	가상 기업과 가상 사건으로 호황·불황·급변 등 다양한 시장 상황을 구성한다.
뉴스·경제 상황의 가격 반영 연습 어려움	사건이 봇의 매매 파라미터를 바꾸고, 같은 사건에도 투자자 유형별 반응이 달라지도록 하여 가격 반영 과정을 연습할 수 있게 한다.
참여 시점 및 일정 제한	하나의 시장에서 기간별 논리적 대회에 자동 편입되어 언제든 참여할 수 있도록 한다.

3. 특징 및 기능

특징

외부 시세 API 없이 종목, 장 운영 시간, 가격 규칙을 직접 설계한 자체 가상 주식장이라는 점이 가장 큰 특징이다. 사용자의 주문과 AI 투자자 봇의 주문이 같은 호가창에서 같은 규칙으로 체결되므로, 사용자의 주문이 실제로 가격을 움직인다. AI 투자자 봇은 페르소나별 확률 규칙에 따라 주문을 내고, 그 성향은 외부 LLM이 장중 2시간 단위로 산출해 검증을 거친 파라미터로 갱신된다. 금리·환율 같은 거시 환경과 실적 발표·경영진 교체 같은 개별 사건이 투자자 유형별로 다르게 반영되어, 같은 사건에도 서로 다른 매매와 가격 흐름이 나타난다. 대회는 별도로 개설하지 않고 하나의 계좌와 하나의 시장 위에서 기간별 성과를 집계하는 논리적 단위로 운영하므로, 사용자는 원하는 때에 참여할 수 있다.

- 기능

1. 자체 가상 주식장 : 반도체, IT, 금융, 통신 등 섹터별 가상 기업을 상장하고, 정규장 시간(09:00~15:30)과 일일 가격제한폭(±30%)을 적용한다.
2. C++ 체결 엔진 : 종목별 호가창을 관리하며 가격 우선·시간 우선으로 지정가·시장가 주문을 체결하고, 부분 체결·정정·취소를 처리한다.
3. AI 투자자 봇 : 페르소나별 매수·매도 봇이 백그라운드 스레드로 동작하며, 현재가와 파라미터로 계산한 확률에 따라 주문을 낸다.
4. LLM 기반 봇 파라미터 산출 : 외부 LLM이 장중 2시간 단위로 시장 상황과 사건을 해석하여 페르소나별 매매 파라미터를 산출하고, 검증을 통과한 값만 봇에 반영한다.
5. 사건 체계 : 금리·환율 같은 거시 환경은 실제 과거 흐름을 재현하고, 실적 발표·경영진 교체 같은 개별 사건은 기업 가치와 투자 심리를 바꾸어 투자자 유형별로 다른 반응이 나타나게 한다.
6. 논리적 기간 대회 : 1개월·3개월·6개월·1년 구간의 투자 성과를 순위로 집계하며, 구간 도중 참가자는 참가 시점 기준으로 편입한다.
7. 실시간 웹 클라이언트 : React 화면에서 실시간 시세·호가·체결, 계좌, 기간별 랭킹을 제공한다.
8. 기록과 재현 : 주문·체결 로그와 모든 LLM 입력·출력·파라미터 변경 이력을 저장하여 시장 변화를 추적하고 재현할 수 있게 한다.

3-1. 기능 정보구조도



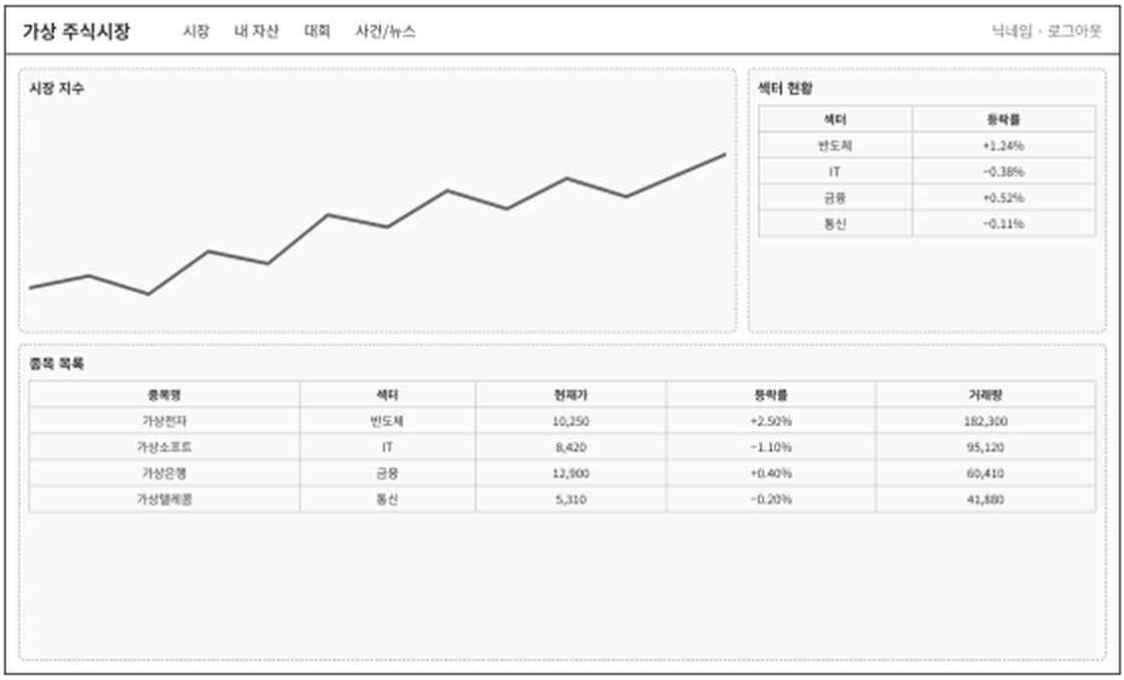
[그림 3-1] 기능 단위 정보구조도

4. 인터페이스 (UI)

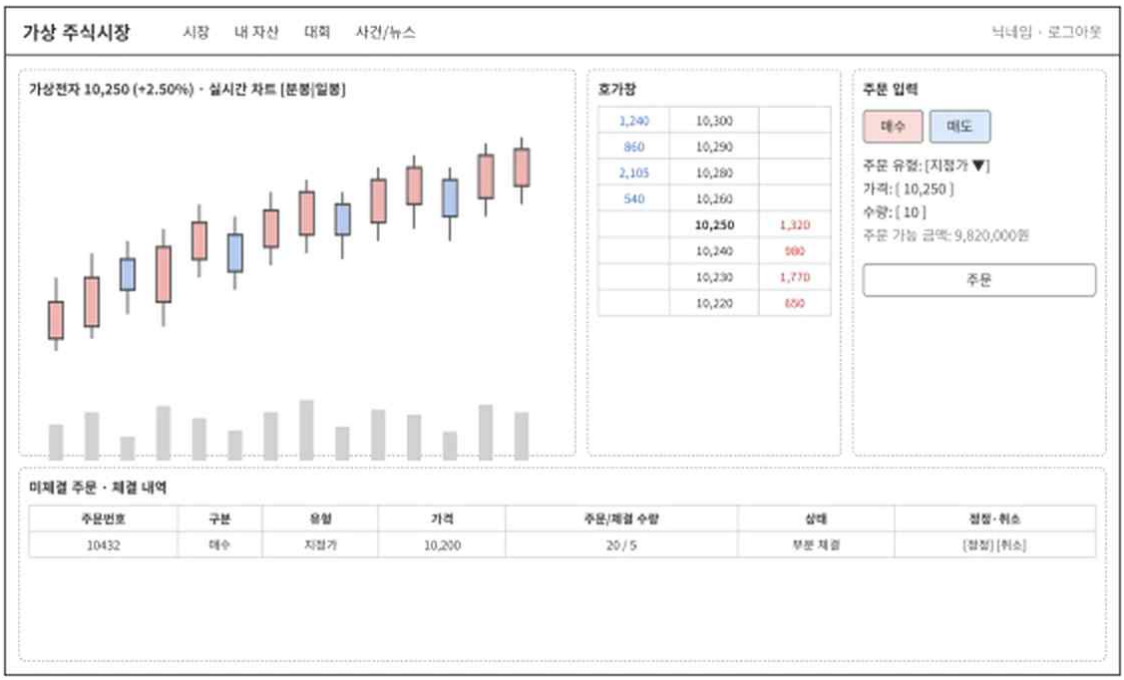
4-1. 페이지 구성

메인 페이지	종목 페이지	내 자산 페이지	대회 페이지	사건/뉴스 페이지	회원 페이지
시장 지수	실시간 차트	보유 종목	기간별 랭킹	사건 타임라인	회원가입/로그인
섹터 현황	호가창	평가손익	내 순위·수익률	사건 상세	회원정보 수정
종목 목록	주문 입력 (지정가/시장가)	주문 내역	지난 구간 결과	섹터별 영향	회원탈퇴/로그아웃

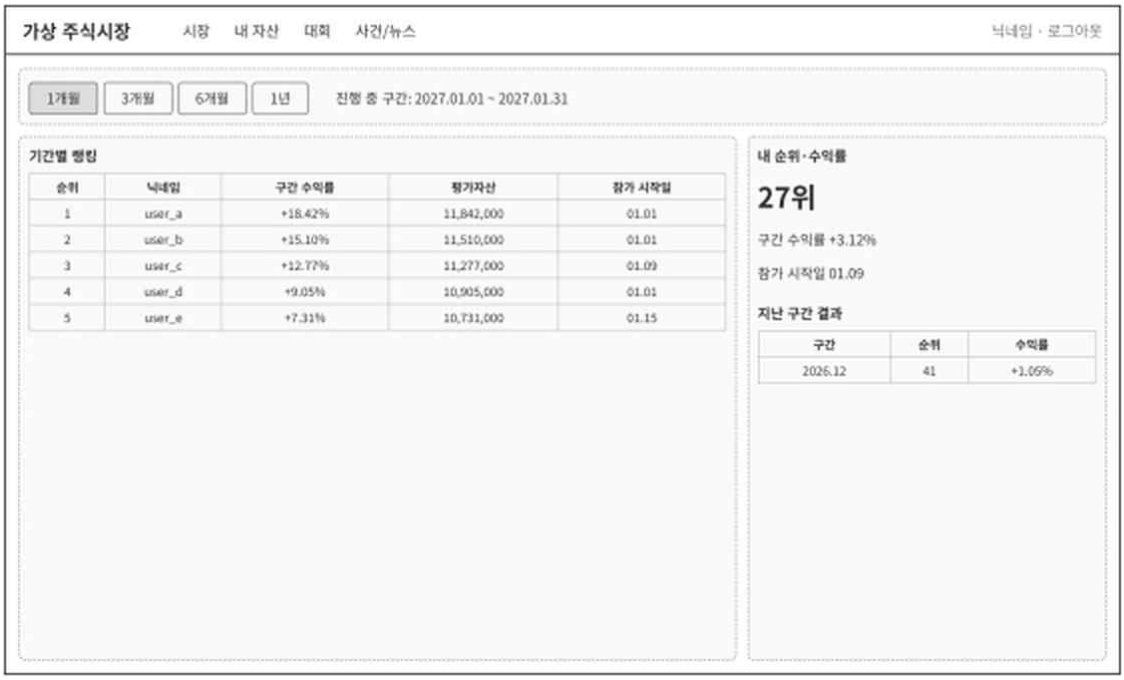
4-2. 주요 화면 시안



[그림 4-1] 메인 페이지 - 시장 지수, 섹터 현황, 종목 목록



[그림 4-2] 종목 페이지 - 실시간 차트, 호가창, 주문 입력, 미체결 주문



[그림 4-3] 대회 페이지 - 기간 선택, 기간별 랭킹, 내 순위, 지난 구간 결과

4-3. 화면 갱신 방식

웹 클라이언트는 React[12]로 구성한다. 계좌·랭킹·사건 조회는 REST API로 처리하고, 시세·호가·체결 알림은 WebSocket(STOMP)을 구독하여 화면을 갱신한다. WebSocket은 저지연·고빈도·대용량 메시지가 필요한 금융 서비스처럼 실시간성이 높은 화면에 적합하다[11].

- 화면 단위 컴포넌트: 시세 차트, 호가창, 주문 입력, 보유 종목 표, 기간별 랭킹 표, 사건 타임라인
- 실시간 데이터: 종목 상세 화면에 진입할 때 해당 종목의 시세·호가를 구독하고, 화면을 벗어나면 구독을 해제한다.

5. 소프트웨어 요구사항 (Software Requirement)

5-1. 기능적 요구사항 (Functional Requirements)

참가자 (User)

1. 회원가입(닉네임 중복 확인), 로그인·로그아웃, 회원정보 수정·탈퇴
2. 시장 지수·섹터 현황·종목 목록 조회
3. 종목별 실시간 시세·차트(분봉·일봉)·호가 조회
4. 매수·매도 주문 (지정가·시장가)
5. 미체결 주문 정정·취소
6. 보유 종목, 평가손익, 예수금, 주문·체결 내역 조회
7. 기간별(1·3·6개월, 1년) 랭킹 및 내 순위·수익률, 지난 구간 결과 조회

8. 사건 타임라인·사건 상세·섹터별 영향 조회

가상 주식시장 (System)

1. 종목별 호가창을 관리하고 가격 우선·시간 우선 원칙으로 주문을 체결한다.
2. 체결되지 않은 잔량을 호가창에 유지하고 정정·취소를 반영한다.
3. 페르소나별 봇이 현재 구간 파라미터에 따라 자동으로 매수·매도 주문을 낸다.
4. 구간 시작 직전 LLM으로 페르소나별 파라미터를 산출하고, 검증·보정 후 반영한다.
5. 거시 환경을 재현하고 개별 사건을 발생시켜 투자자 유형별 심리 충격을 반영한다.
6. 정해진 시각에 개장·폐장, 2시간 구간 전환, 장 마감 처리를 실행한다.
7. 장 마감 시 평가자산 스냅샷을 저장하고 기간 수익률과 순위를 집계한다.
8. 시세·호가·체결 정보를 구독자에게 실시간으로 전송한다.
9. 주문·체결·자산 변동, LLM 입력·출력·적용 이력을 저장한다.

5-2. 비기능적 요구사항 (Non-functional Requirements)

1. 효율성 요구사항 (efficiency requirement)

- 체결 엔진은 성능과 메모리를 직접 제어하기 위해 C++로 구현하고, 종목 단위로 호가창을 분리하여 종목별 하나의 처리 흐름에서 순차 처리한다. 잠금 경합을 줄이고 결과를 재현 가능하게 한다.
- LLM은 구간 시작 전 한 번만 호출하므로 LLM의 응답 시간과 비용이 체결 처리 속도에 영향을 주지 않는다.
- 봇은 스레드 풀 위에서 각자의 호출 주기로 동작하며, 주문 속도 제한으로 엔진에 과부하가 걸리지 않게 한다.
- 진행 중인 구간의 순위는 Redis Sorted Set으로 O(log N)에 갱신·조회한다 [10].
- 다수의 봇으로 부하 시험을 수행하여 처리량과 지연 시간을 측정하고, 결과에 따라 파라미터를 조정한다.

2. 신뢰성 요구사항 (reliability requirement)

- 주문 접수 시 주문 금액(매수) 또는 보유 수량(매도)을 묶어 두어, 동시 주문에서도 현금과 주식이 초과 사용되지 않게 한다.
- 모든 주문·체결을 순서대로 로그로 남기고, 엔진 장애 후 로그를 재생하여 호가창을 복원한다.
- LLM 응답은 형식 검사, 값 범위 보정, 1회 변경 폭 제한을 거친 값만 반영한다. 호출 실패·시간 초과·이상값 발생 시 이전 구간 파라미터를 유지한다.
- 신규 파라미터를 신규 반영 비율만큼만 봇 인스턴스에 적용하고 일일 가격제한폭(±30%)을 두어 가격 폭주·붕괴를 방지한다.

3. 사용자 요구사항 (usability requirement)

- 주요 기능을 메인·종목·내 자산·대회·사건/뉴스·회원의 6개 페이지로 나누어 제공한다.
- 오류 상황: 닉네임 중복, 입력 형식 오류, 주문 가능 금액·보유 수량 부족, 가격제한폭·호가 단위 위반, 장 운영 시간 외 주문은 각각 오류 메시지를 표시한다.
- 구간 도중 참가자의 형평성을 위해 랭킹에 구간별 참가 시작일을 함께 표시한다.

4. 이식성 요구사항 (portability requirement)

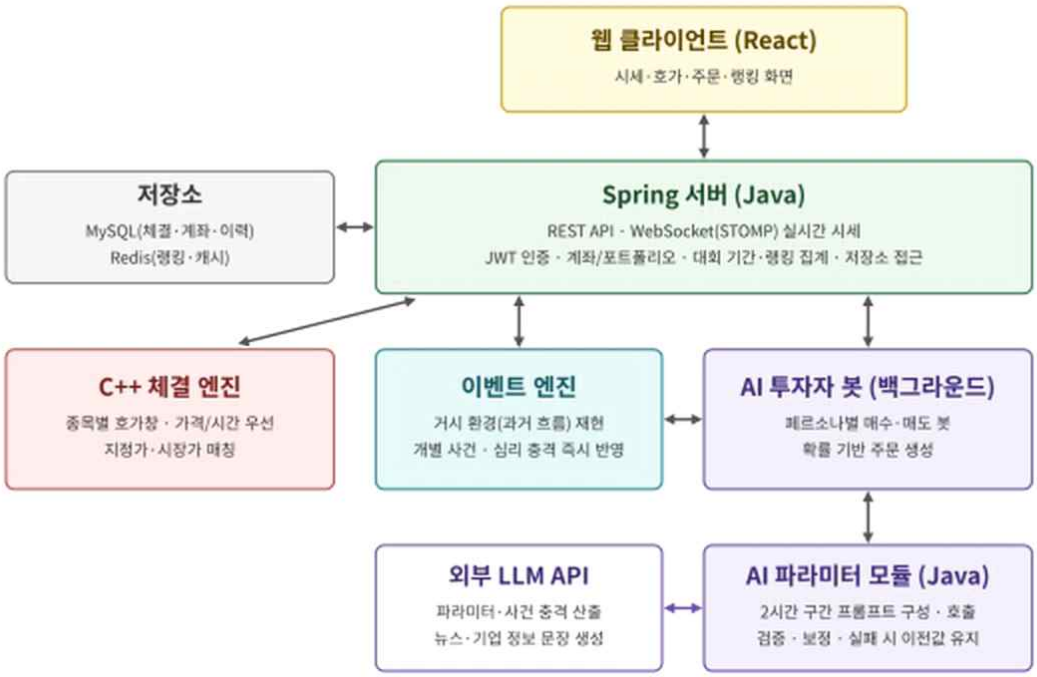
- 웹 브라우저에서 동작하는 React 클라이언트로 제공하여 별도 설치 없이 사용할 수 있게 한다.
- 서버 실행 환경은 Docker Compose로 구성하고 접속 정보를 환경 변수로 분리하여, 개발자마다 같은 환경을 재현할 수 있게 한다.
- 시장 단위로 엔진과 봇을 분리하여 이후 여러 시장으로 확장할 수 있도록 한다.

5. 보안 요구사항 (security requirement)

- 로그인 상태는 JWT로 관리하며, 모든 REST API 요청과 WebSocket 연결을 같은 방식으로 인증한다[14].
- 비밀번호는 해시하여 저장한다.
- 개인 체결 알림은 사용자별 큐(/user/queue/executions)로 본인에게만 전송한다.

6. 시스템 대블록 설계

시스템은 웹 클라이언트, Spring 서버, C++ 체결 엔진, 이벤트 엔진, AI 투자자 봇, AI 파라미터 모듈, 저장소로 구성된다. 사용자와 봇의 주문은 모두 Spring 서버를 거쳐 C++ 체결 엔진으로 전달되고, 체결 결과는 저장소에 기록된 뒤 실시간으로 클라이언트에 전송된다. AI 파라미터 모듈은 구간마다 외부 LLM API를 호출해 봇 파라미터와 사건 충격을 산출한다.

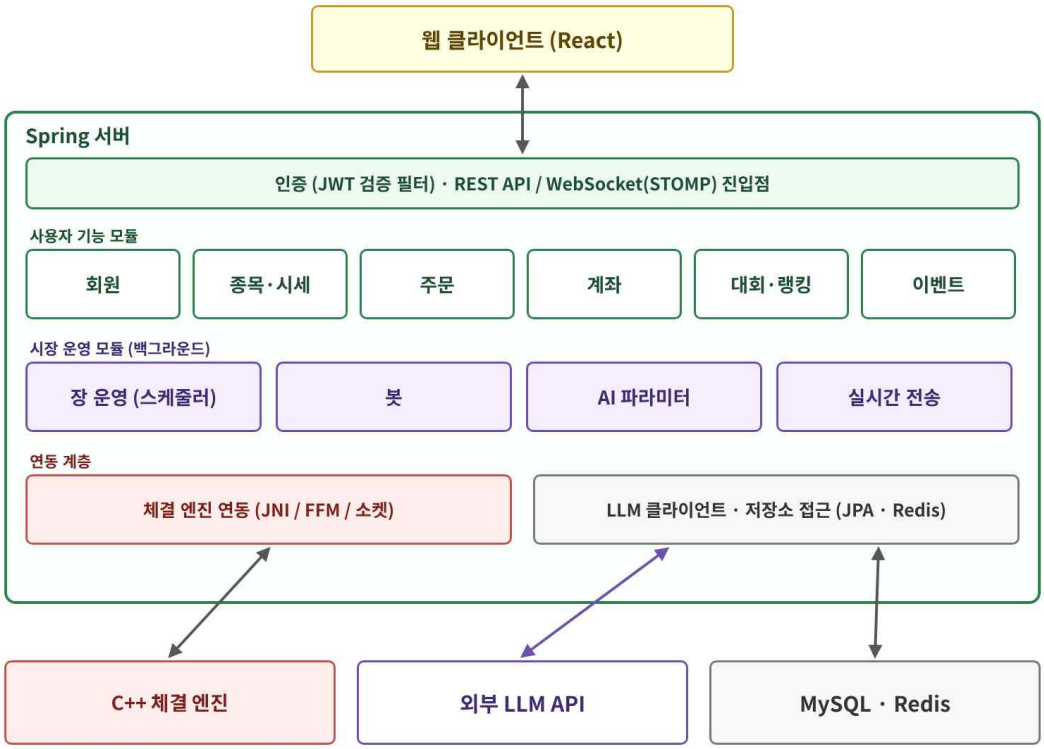


[그림 6-1] 시스템 구성도

구성 요소	기술	역할
웹 클라이언트	React[12]	시세·호가·주문·계좌·랭킹 화면, WebSocket 구독으로 실시간 갱신
체결 엔진	C++	종목별 호가창 관리, 가격/시간 우선 매칭, 체결 결과 반환
메인 서버	Java, Spring[11]	REST API, WebSocket(STOMP) 실시간 시세, JWT 인증, 계좌·포트폴리오, 대회 기간·랭킹 집계
AI 투자자 봇	Java 백그라운드 스레드	페르소나별 매수·매도 봇, 확률 기반 주문 생성, 2시간 구간 파라미터 보관
이벤트 엔진	Java	사건 일정 관리, 거시 환경(과거 흐름) 재현, 개별 사건 발생과 투자 심리 충격 반영
AI 파라미터 모듈	Java, 외부 LLM API	2시간 구간마다 LLM으로 페르소나별 파라미터 산출, 응답 검증·보정, 실패 시 이전값 유지, 사건 충격 산출, 뉴스 문장 생성
저장소	MySQL, Redis[10]	체결·계좌·이력 영속 저장, 랭킹·시세 캐시

7. 모듈 기반 소블록 인터페이스

서버는 기능 단위 모듈로 나누어 구현한다. 각 모듈은 도메인 패키지(controller·service·repository·entity·dto)로 구성하고, C++ 체결 엔진과 외부 LLM 같은 외부 시스템 연동은 별도의 연동 계층에 둔다.



[그림 7-1] 서버 모듈 구성

7-1. 모듈별 책임과 인터페이스

모듈	책임	주요 인터페이스	관련 UC
인증	회원가입, 로그인, JWT 발급·검증, 요청 인증 필터	POST /users GET /users/nicknames/{nickname}/availability POST /auth/login	UC-01
회원	회원 정보 조회·수정·탈퇴	GET·PATCH·DELETE /users/me	UC-02, 03
종목·시세	종목·섹터 정보, 현재가, 분봉·일봉 집계, 호가 스냅샷	GET /stocks GET /stocks/{stockId} GET /stocks/{stockId}/candles	UC-04, 05, 06
주문	주문 검증(가격제한폭·잔고·보유 수량), 접수, 정정·취소, 체결 결과 반영	POST /orders PATCH·DELETE /orders/{orderId} GET /users/me/orders	UC-07, 08, 09, 11, 14
계좌	예수금, 보유 종목, 평가손익, 일별 평가자산 스냅샷	GET /users/me/account GET /users/me/stocks	UC-10, 17
체결 엔진 연동	C++ 체결 엔진으로 주문 전달, 체결 결과 수신	내부 인터페이스	UC-07, 08, 09, 14
실시간 전송	WebSocket(STOMP) 연결, 시세·호가·체결 메시지 발행	/ws /topic/stocks/{stockId} /user/queue/executions	UC-05, 06, 07
봇	페르소나·봇 인스턴스 관리, 호출 스케줄링, 확률 기반 주문 생성, 주문 속도 제한	내부 인터페이스	UC-14
AI 파라미터	LLM 프롬프트 구성·호출, 응답 검증·보정, 산출 이력 저장	내부 인터페이스, 외부 LLM API	UC-15, 16
이벤트	사건 일정·발생, 거시 환경 재현, 사건 충격 반영, 뉴스 생성	GET /events GET /events/{eventId}	UC-13, 16
대회·랭킹	기간 구간 관리, 구간 수익률 계산, Redis 순위 집계	GET /rankings?period= GET /rankings/me	UC-12, 17
장 운영	개장·폐장, 2시간 구간 전환, 장 마감 처리 실행	내부 스케줄러	UC-15, 16, 17

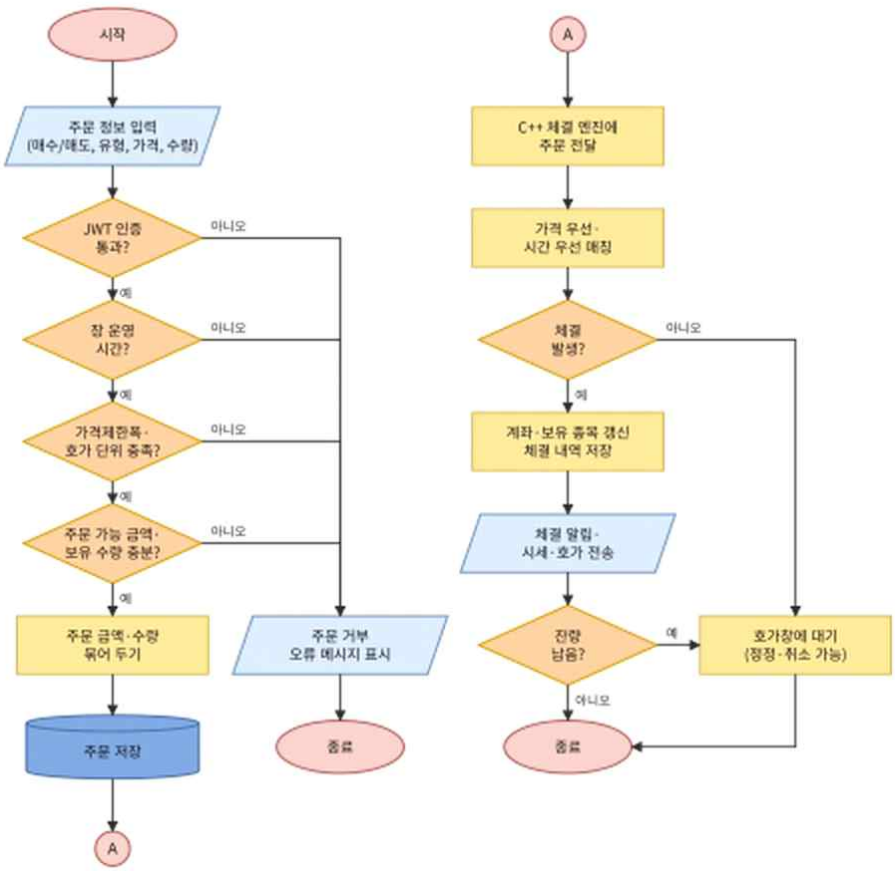
7-2. 주요 모듈 간 연결

호출자	피호출자	방식	전달 데이터
모든 API	인증	요청 전 필터	JWT
주문	계좌	동기 호출	주문 가능 금액·수량 확인 및 묶음 처리
주문, 봇	체결 엔진 연동	동기 호출	주문(종목, 매수/매도, 가격, 수량, 주문자)
체결 엔진 연동	주문, 계좌	체결 이벤트	주문 ID, 체결 가격·수량
체결 엔진 연동	종목·시세, 실시간 전송	체결 이벤트	현재가, 호가 변화
장 운영	AI 파라미터 → 봇	스케줄 실행	구간 시작 시각, 검증된 파라미터
이벤트	AI 파라미터 → 봇	사건 이벤트	사건 내용, 페르소나별 충격(기준 가격 이동, 지연·지속 시간)
장 운영	계좌 → 대회·랭킹	스케줄 실행	장 마감 평가자산 스냅샷

8. 구성도 (Flow Chart)

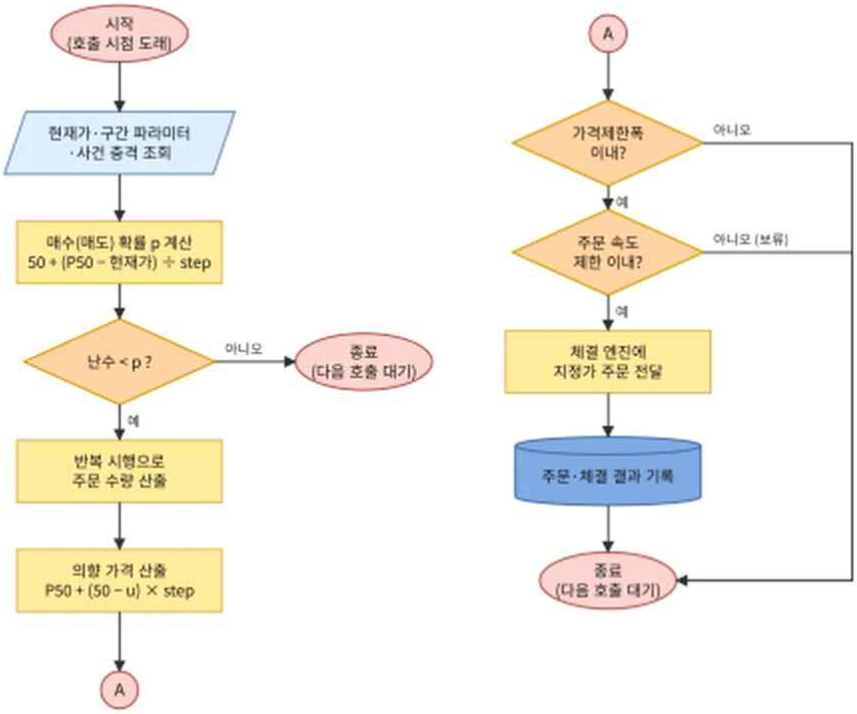
시스템의 핵심 흐름인 참가자 주문 처리, 봇 자동 주문, LLM 파라미터 갱신을 순서도로 나타낸다. 원 안의 같은 문자는 서로 이어지는 연결점이다.

8-1. 상세 - 주문 처리 구성도



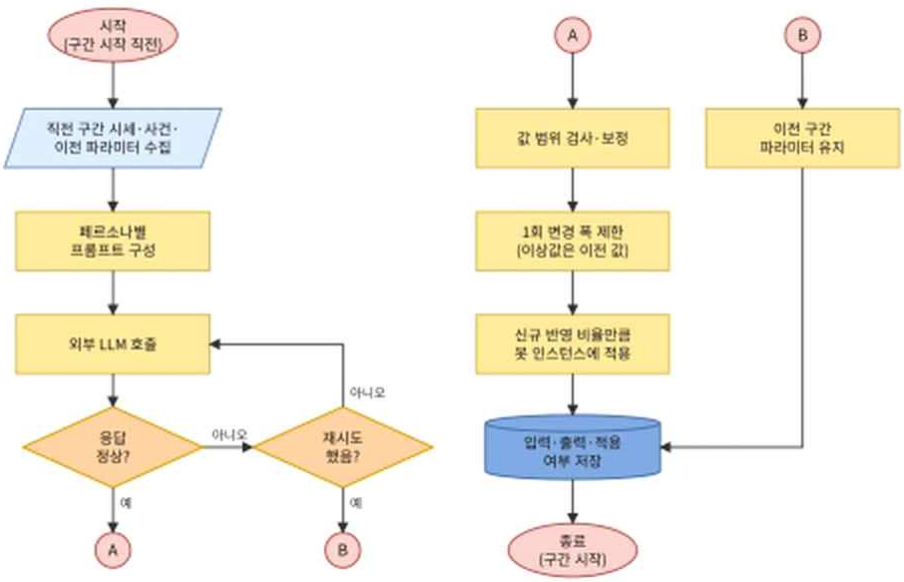
[그림 8-1] 참가자 주문 처리 흐름 (UC-07)

8-2. 상세 - 봇 자동 주문 구성도



[그림 8-2] AI 투자자 봇 주문 생성 흐름 (UC-14)

8-3. 상세 - LLM 파라미터 갱신 구성도



[그림 8-3] 구간별 봇 파라미터 갱신 흐름 (UC-15)

정규장(09:00~15:30)은 09:00~11:00, 11:00~13:00, 13:00~15:30의 세 구간으로 나누고, 각 구간이 시작되기 직전에 위 흐름을 실행한다. 구간 도중 발생한 사건은 다음 구간을 기다리지 않고 사건 충격으로 즉시 반영한다.



[그림 8-4] 장중 2시간 구간별 LLM 호출 시점

9. 유즈케이스 다이어그램 (Usecase Diagram)

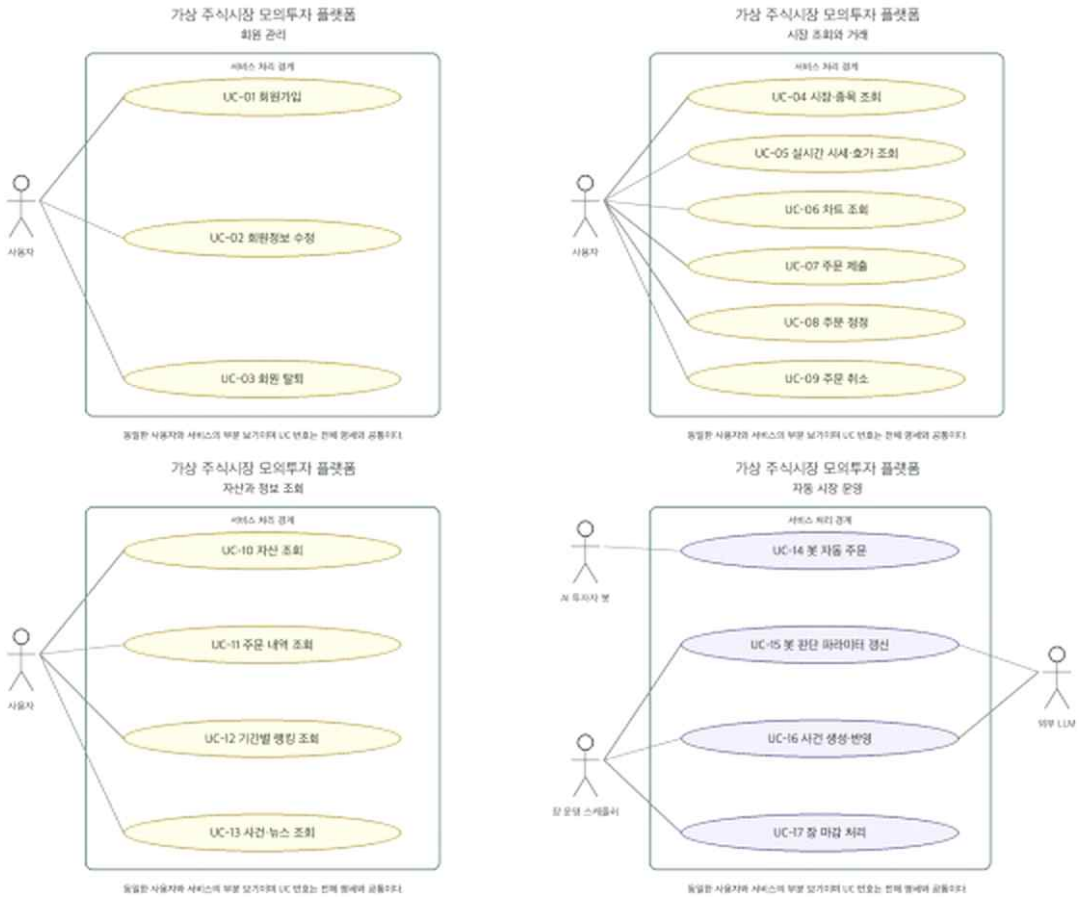
3장의 기능을 사용자 관점의 유즈케이스로 정리하고, 유즈케이스마다 담당 모듈을 정해 두어 이후 상세 설계와 구현이 같은 기준을 따르도록 한다.

9-1. 액터

액터	구분	설명
사용자	주 액터	회원가입 후 시세와 차트를 보고 주문하며, 자산·주문 내역과 기간별 순위를 확인한다.
AI 투자자 봇	시스템 액터	현재 구간 파라미터에 따라 자동으로 매수·매도 주문을 낸다.
장 운영 스케줄러	시스템 액터	개장·폐장, 2시간 구간 전환, 장 마감 처리를 정해진 시각에 실행한다.
외부 LLM	외부 시스템	봇 판단 파라미터와 사건 충격을 산출하고, 뉴스 문장을 생성한다.

9-2. 유즈케이스 다이어그램

유즈케이스를 회원 관리, 시장 조회와 거래, 자산과 정보 조회, 자동 시장 운영의 네 영역으로 나누어 나타낸다. 네 그림은 동일한 사용자와 서비스의 부분 보기이며, UC 번호는 전체 명세와 공통이다.



[그림 9-1] 영역별 유즈케이스 다이어그램

9-3. 유즈케이스 목록

ID	유즈케이스	액터	영역	관련 화면·기능
UC-01	회원가입	사용자	회원 관리	회원 페이지
UC-02	회원정보 수정	사용자	회원 관리	회원 페이지
UC-03	회원 탈퇴	사용자	회원 관리	회원 페이지
UC-04	시장·종목 조회	사용자	시장 조회와 거래	메인 페이지
UC-05	실시간 시세·호가 조회	사용자	시장 조회와 거래	종목 페이지
UC-06	차트 조회	사용자	시장 조회와 거래	종목 페이지
UC-07	주문 제출	사용자	시장 조회와 거래	종목 페이지
UC-08	주문 정정	사용자	시장 조회와 거래	종목·내 자산 페이지
UC-09	주문 취소	사용자	시장 조회와 거래	종목·내 자산 페이지
UC-10	자산 조회	사용자	자산과 정보 조회	내 자산 페이지
UC-11	주문 내역 조회	사용자	자산과 정보 조회	내 자산 페이지
UC-12	기간별 랭킹 조회	사용자	자산과 정보 조회	대회 페이지
UC-13	사건·뉴스 조회	사용자	자산과 정보 조회	사건/뉴스 페이지
UC-14	봇 자동 주문	AI 투자자 봇	자동 시장 운영	AI 투자자 봇
UC-15	봇 판단 파라미터 갱신	장 운영 스케줄러, 외부 LLM	자동 시장 운영	LLM 2시간 갱신
UC-16	사건 생성·반영	장 운영 스케줄러, 외부 LLM	자동 시장 운영	사건 엔진
UC-17	장 마감 처리	장 운영 스케줄러	자동 시장 운영	기간 수익률 집계

9-4. 핵심 유즈케이스 명세

UC-01 회원가입	내용
액터	사용자
사전 조건	로그인하지 않은 상태
기본 흐름	닉네임 입력 → 중복 확인 요청 → 사용 가능 표시 → 비밀번호 입력 후 가입 요청 → 시스템이 입력 형식과 닉네임 중복을 다시 확인 → 비밀번호를 해시하여 저장하고 시드머니 지급 → 진행 중인 기간 대회에 자동 편입
대안 흐름	닉네임이 이미 사용 중이면 오류 메시지를 표시하고 다른 닉네임을 입력받는다. 입력 형식이 맞지 않으면 항목별 오류를 표시한다.
사후 조건	회원과 계좌가 생성되고, 해당 닉네임과 비밀번호로 로그인할 수 있다.

UC-07 주문 제출	내용
액터	사용자
사전 조건	로그인 상태(JWT 보유), 장 운영 시간
기본 흐름	종목 화면에서 매수/매도, 주문 유형(지정가/시장가), 가격, 수량 입력 → 주문 요청 → 서버가 가격제한폭·호가 단위와 주문 가능 금액(매수) 또는 보유 수량(매도)을 확인 → 주문 금액·수량을 묶어 두고 체결 엔진에 전달 → 체결되면 계좌와 보유 종목을 갱신하고 체결 알림 전송 → 시세·호가 변화를 구독자에게 전송
대안 흐름	잔고나 보유 수량이 부족하거나 가격제한폭을 벗어나면 주문을 거부한다. 장 운영 시간이 아니면 주문을 받지 않는다. 체결되지 않은 잔량은 호가창에 남고 UC-08(주문 정정)·UC-09(주문 취소)로 처리할 수 있다.
사후 조건	주문과 체결 내역이 저장되고, 체결 결과가 가격에 반영된다.

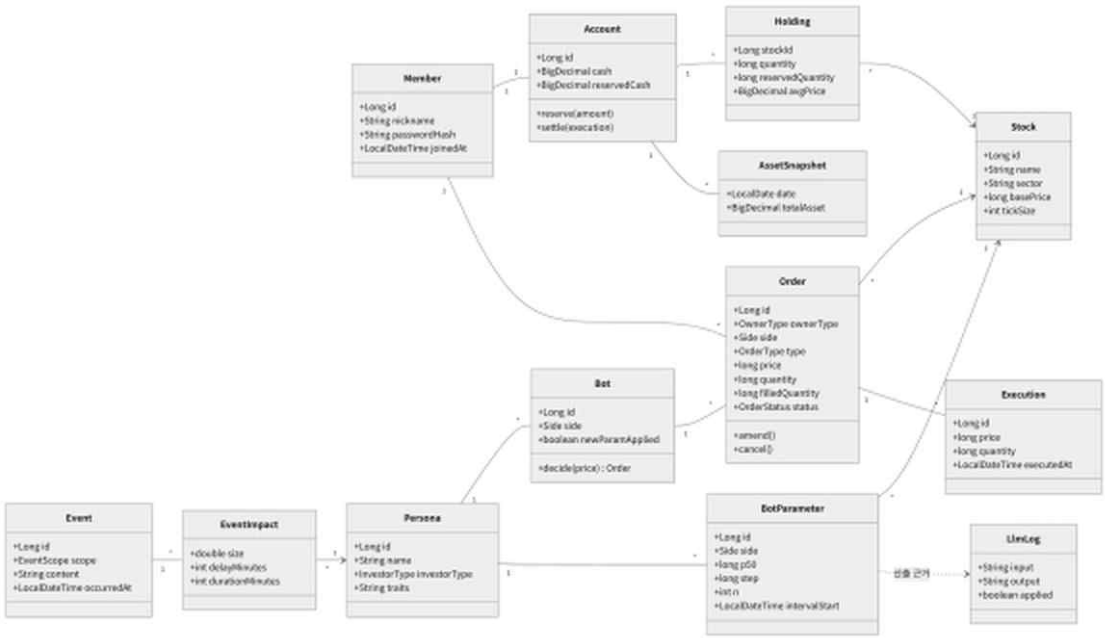
UC-14 봇 자동 주문	내용
액터	AI 투자자 봇
사전 조건	장 운영 시간, 현재 구간 파라미터가 적용된 상태
기본 흐름	호출 시점 도래(평균 60 ÷ N초) → 현재가 조회 → 매수(매도) 확률 계산 → 난수로 주문 여부 결정 → 의향 가격과 수량 산출 → 주문 속도 제한 확인 → 체결 엔진에 주문 전달 → 주문 결과 기록
대안 흐름	주문하지 않기로 결정되면 다음 호출을 기다린다. 속도 제한을 넘으면 주문을 보류한다. 의향 가격이 가격제한폭을 벗어나면 주문하지 않는다.
사후 조건	봇 주문과 체결 내역이 주문 테이블에 저장된다.

UC-15 봇 판단 파라미터 갱신	내용
액터	장 운영 스케줄러, 외부 LLM
사전 조건	다음 2시간 구간이 시작되기 직전
기본 흐름	스케줄러가 갱신 작업 실행 → 직전 구간 시세·사건·이전 파라미터 수집 → 페르소나별 프롬프트 구성 → LLM 호출 → 응답 형식 검사 → 값 범위 보정과 변경 폭 제한 → 신규 반영 비율에 따라 봇 인스턴스에 적용 → 입력·출력·적용 여부 저장
대안 흐름	호출 실패·시간 초과·형식 오류가 나면 한 번 다시 시도하고, 그래도 실패하면 이전 구간 파라미터를 유지한다. 일부 값만 이상하면 해당 값만 이전 값으로 유지한다.
사후 조건	새 구간 파라미터가 적용되고 갱신 이력이 남는다.

10. 클래스 다이어그램 (Class Diagram)

10-1. 도메인 클래스 다이어그램

서버의 핵심 도메인 클래스와 관계를 나타낸다. 회원 주문과 봇 주문은 같은 주문(Order) 클래스로 다루고 주문자 구분(ownerType)으로 구별한다. 봇 파라미터는 구간마다 새로 저장하여 변경 이력을 남긴다.

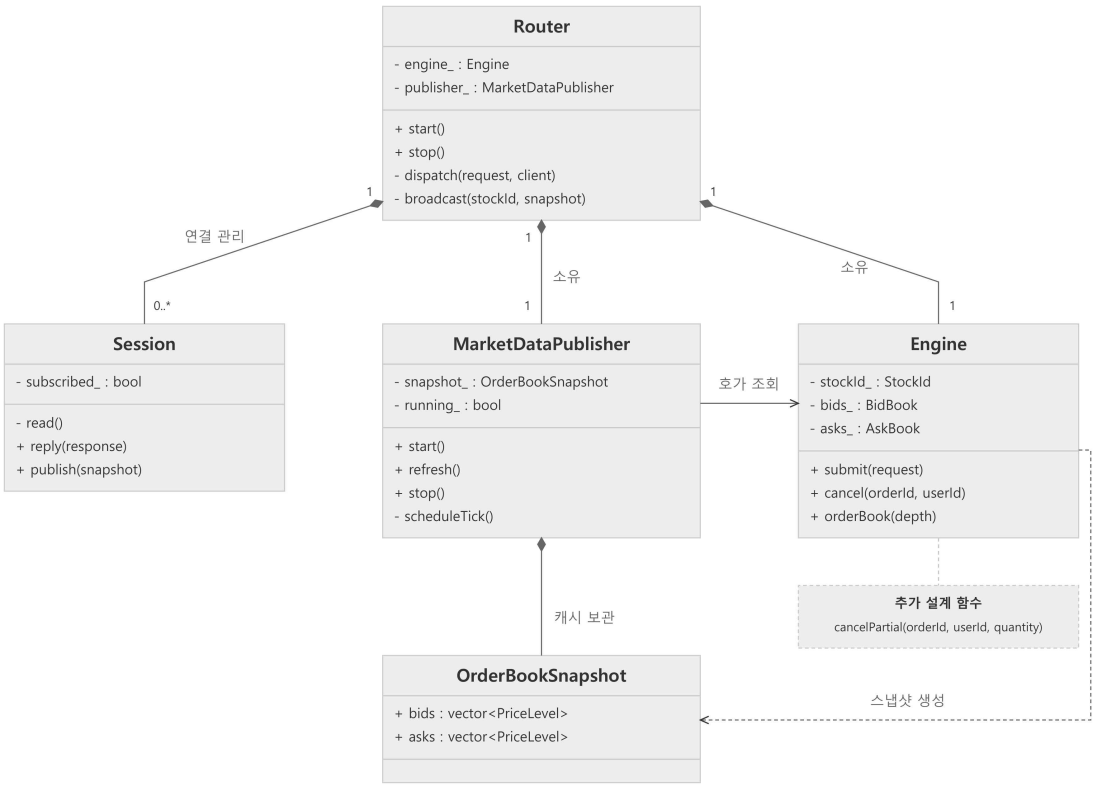


[그림 10-1] 도메인 클래스 다이어그램

클래스	설명
Member / Account	회원과 계좌. 계좌는 예수금과 주문 시 묶어 둔 금액(reservedCash)을 분리해 관리한다.
Holding	계좌별 보유 종목. 매도 주문 시 묶어 둔 수량(reservedQuantity)을 함께 관리한다.
Stock	가상 종목. 섹터, 기준가격, 호가 단위를 가진다.
Order / Execution	회원·봇 공용 주문과 체결 내역. 주문은 부분 체결 수량과 상태를 가진다.
Persona / Bot	페르소나 정의와 페르소나별 매수·매도 봇 인스턴스. 봇은 신규 파라미터 반영 여부를 가진다.
BotParameter	구간별·종목별 페르소나 파라미터(P50, step, N)와 적용 구간.
Event / EventImpact	사건과 페르소나별 충격(크기, 반응 지연, 지속 시간).
AssetSnapshot	장 마감 시점의 일별 평가자산. 기간 수익률 계산의 기준이 된다.
LlmLog	LLM 입력·출력, 검증 결과, 적용 여부.

10-2. C++ 체결 엔진 클래스 구조

앞의 도메인 클래스와 연계되는 C++ 체결 엔진의 주요 클래스 및 관계를 나타낸다. Router는 Session의 요청을 엔진 Worker로 전달하고, Engine은 주문 체결·취소와 호가 조회를 수행한다. MarketDataPublisher는 호가 스냅샷을 보관하고 주기적으로 발행한다.



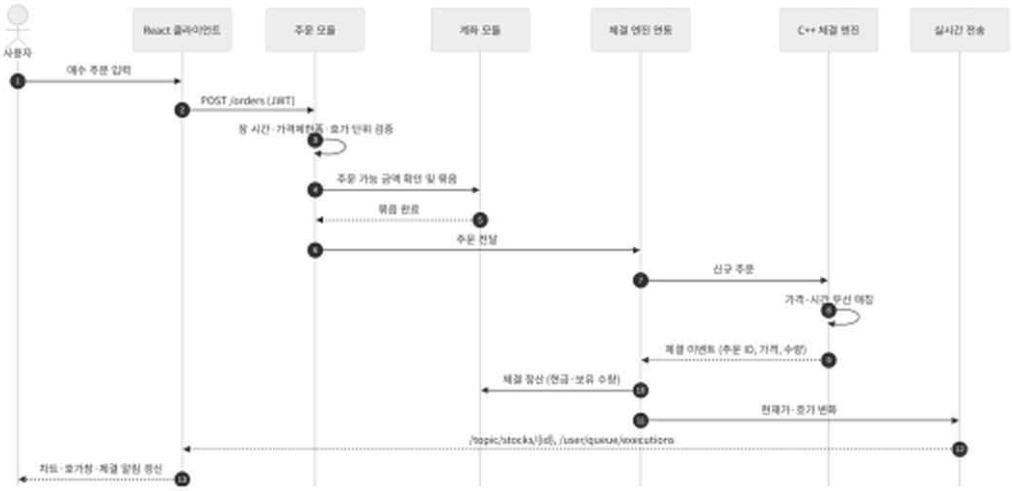
[그림 10-2] C++ 체결 엔진 클래스 다이어그램

구조를 간략히 표현하기 위해 Router::Impl과 Engine::Impl의 멤버를 상위 클래스에 합쳐 표시하였다. 그림의 cancelPartial()은 미체결 수량 감소를 위한 추가 설계 함수이며, 현재 구현 범위는 주문 제출·잔량 전체 취소·호가 조회와 발행이다.

11. 시퀀스 다이어그램 (Sequence Diagram)

11-1. 참가자 주문 체결

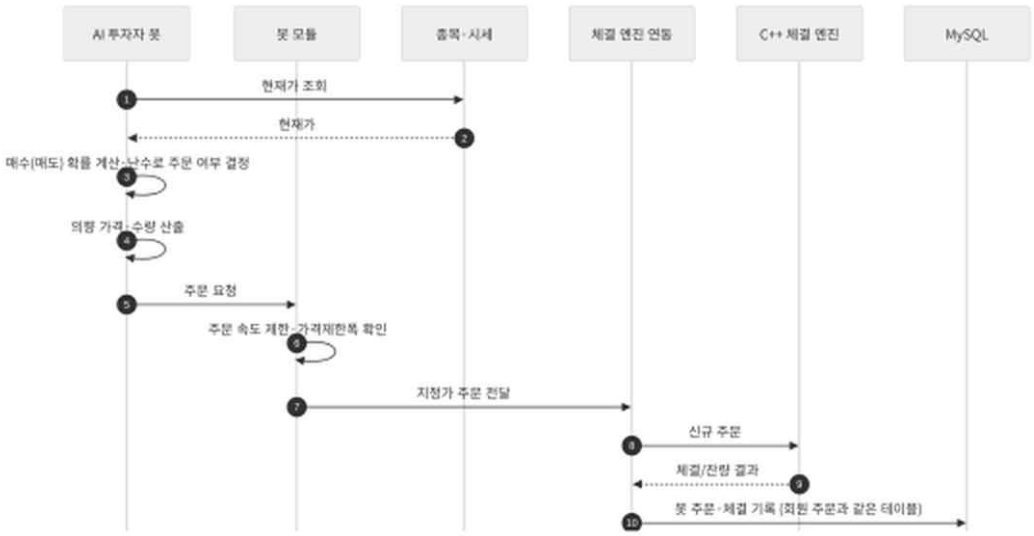
참가자의 주문이 검증·묶음 처리를 거쳐 체결 엔진에서 매칭되고, 체결 결과가 계좌와 화면에 반영되는 순서이다.



[그림 11-1] 참가자 주문 체결 시퀀스

11-2. 봇 자동 주문

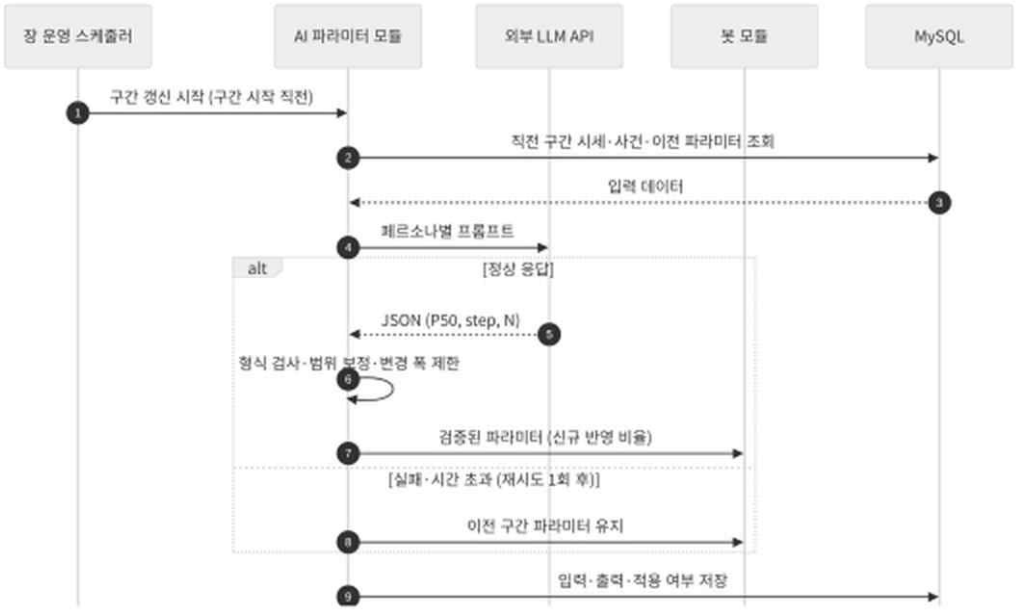
봇이 호출 시점마다 확률로 주문 여부를 정하고, 회원 주문과 같은 경로로 체결 엔진에 주문을 전달하는 순서이다.



[그림 11-2] 봇 자동 주문 시퀀스

11-3. 봇 파라미터 갱신

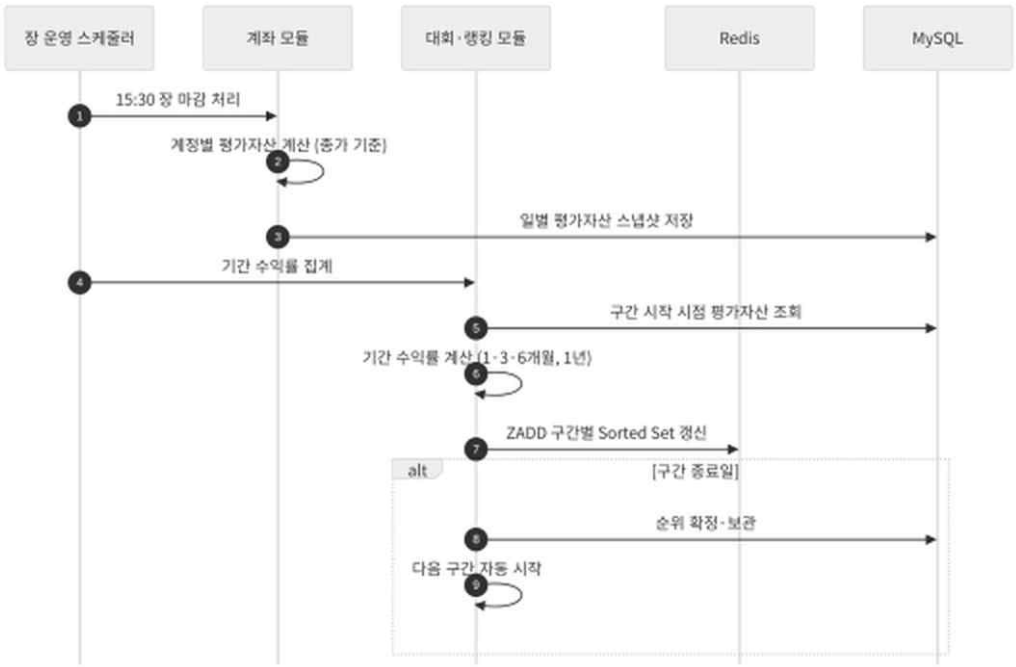
구간 시작 직전 LLM을 호출해 파라미터를 산출하고, 검증 결과에 따라 신규 값 또는 이전 값을 봇에 적용하는 순서이다.



[그림 11-3] 봇 파라미터 갱신 시퀀스

11-4. 장 마감 처리

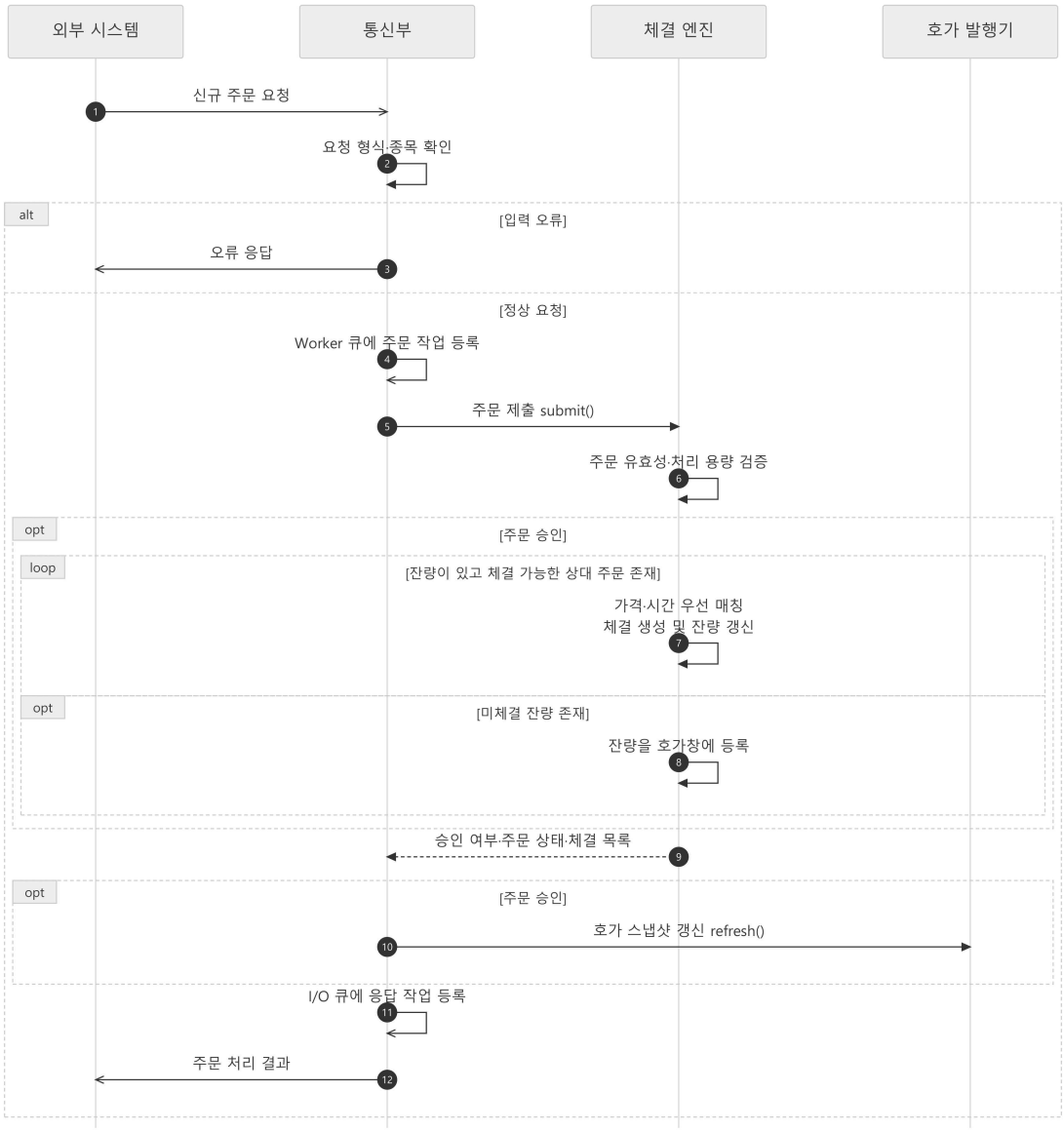
장 마감 시 평가자산 스냅샷을 저장하고 기간 수익률과 순위를 갱신하며, 구간 종료일에는 순위를 확정하는 순서이다.



[그림 11-4] 장 마감 처리 시퀀스

11-5. C++ 엔진 내부 주문 제출 및 체결 (UC-07)

11-1절의 전체 주문 처리 중 C++ 엔진에 요청이 도착한 이후의 내부 흐름이다. 요청을 검증한 뒤 가격·시간 우선으로 상대 주문과 매칭하고, 미체결 잔량을 호가창에 등록한다. 복수 체결 내역은 주문 결과에 모아 반환한다.

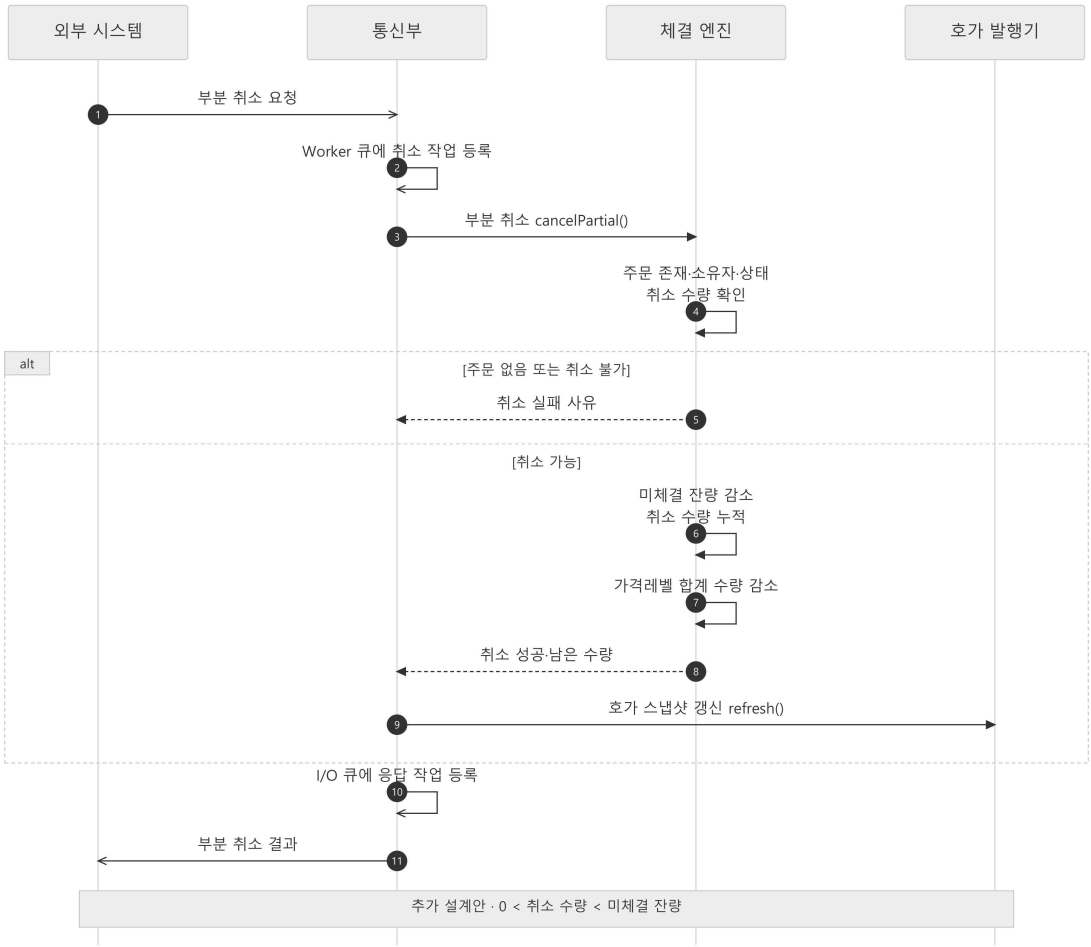


[그림 11-5] 주문 제출 및 체결 시퀀스 (UC-07)

그림 11-5~11-8의 통신부는 Session과 Router::Impl을, 호가 발행기는 MarketDataPublisher를 뜻한다. Worker 큐는 engineContext_, I/O 큐는 ioContext_이며, 열린 화살촉은 비동기 전달, 실선의 채운 화살촉은 동기 호출, 점선은 결과 반환을 나타낸다.

11-6. 주문 정정 중 부분 취소 (UC-08, 추가 설계)

미체결 수량을 줄이는 정정은 기존 주문의 잔량 일부를 취소하는 방식으로 설계한다. 주문 존재 여부·소유자·상태와 취소 수량을 확인한 뒤 주문 잔량과 가격레벨 합계 수량을 줄이고, 결과와 남은 수량을 반환한다.

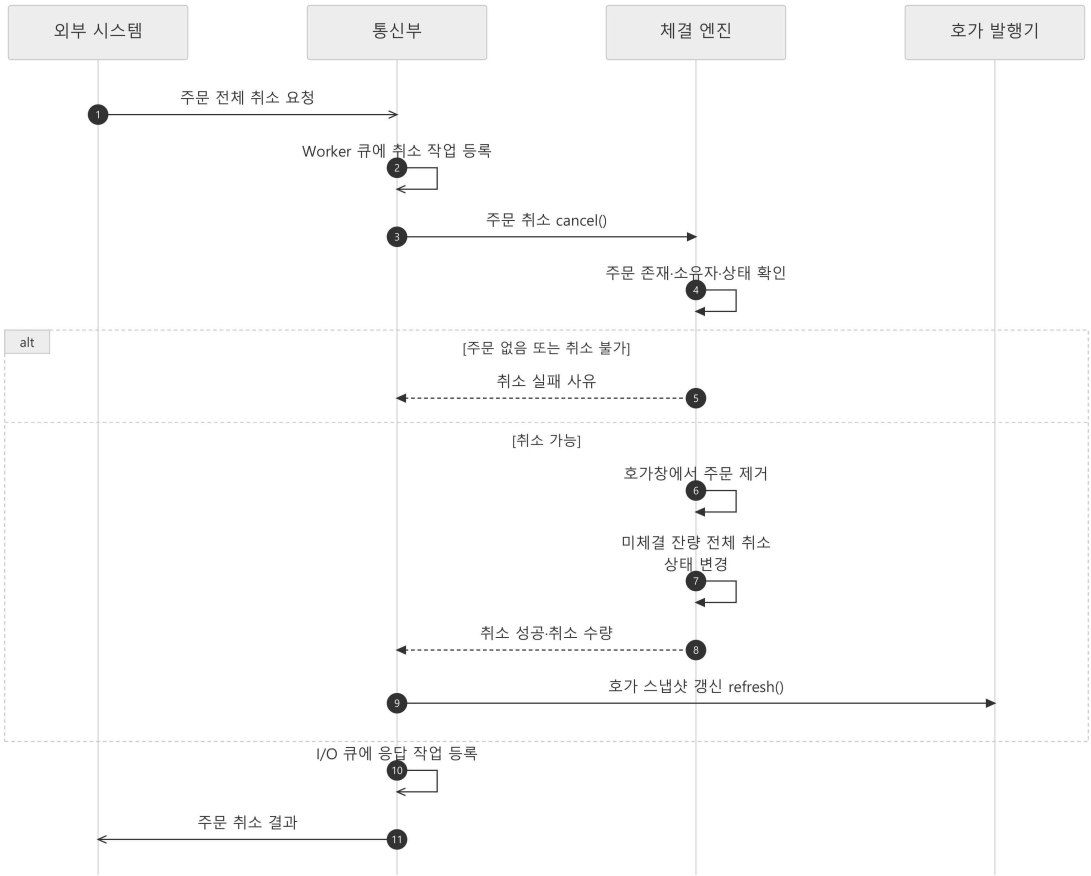


[그림 11-6] 주문 정정 중 부분 취소 시퀀스 (UC-08 추가 설계안)

부분 취소의 수량은 0보다 크고 현재 미체결 잔량보다 작아야 한다. 주문 ID·가격·기존 시간 우선순위를 유지하며, 잔량 전체를 취소하는 경우에는 11-7절의 전체 취소 흐름을 적용한다. 이 기능은 현재 구현에 추가할 설계안이다.

11-7. 주문 전체 취소 (UC-09)

주문 ID로 대상을 조회하고 소유자와 주문 상태를 확인한 뒤, 미체결 잔량 전체를 취소하는 순서이다. 취소 가능한 주문은 호가창에서 제거하고 상태를 Canceled로 변경하며, 호가 스냅샷을 갱신한 후 취소 수량을 반환한다.

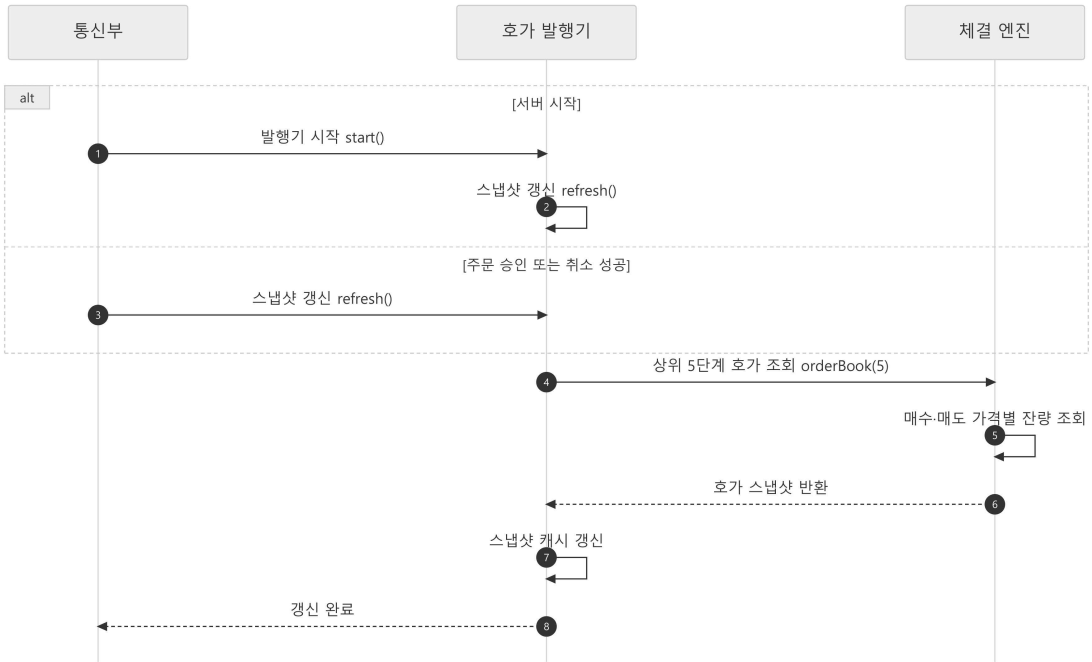


[그림 11-7] 주문 전체 취소 시퀀스 (UC-09)

부분 체결 주문도 남아 있는 수량은 취소할 수 있다. 이미 전량 체결되었거나 취소된 주문은 실패 사유를 반환한다. 취소와 체결 요청이 경쟁하면 엔진 Worker에서 먼저 처리된 작업의 결과가 이후 요청에 반영된다.

11-8. 호가 스냅샷 생성 및 갱신

서버 시작 시와 주문 승인·취소 성공 직후에 매수·매도 상위 5개 가격레벨을 조회하여 호가 스냅샷을 갱신하는 순서이다. 각 가격레벨에 유지되는 미체결 잔량 합계를 복사하여 Publisher의 캐시에 저장한다.

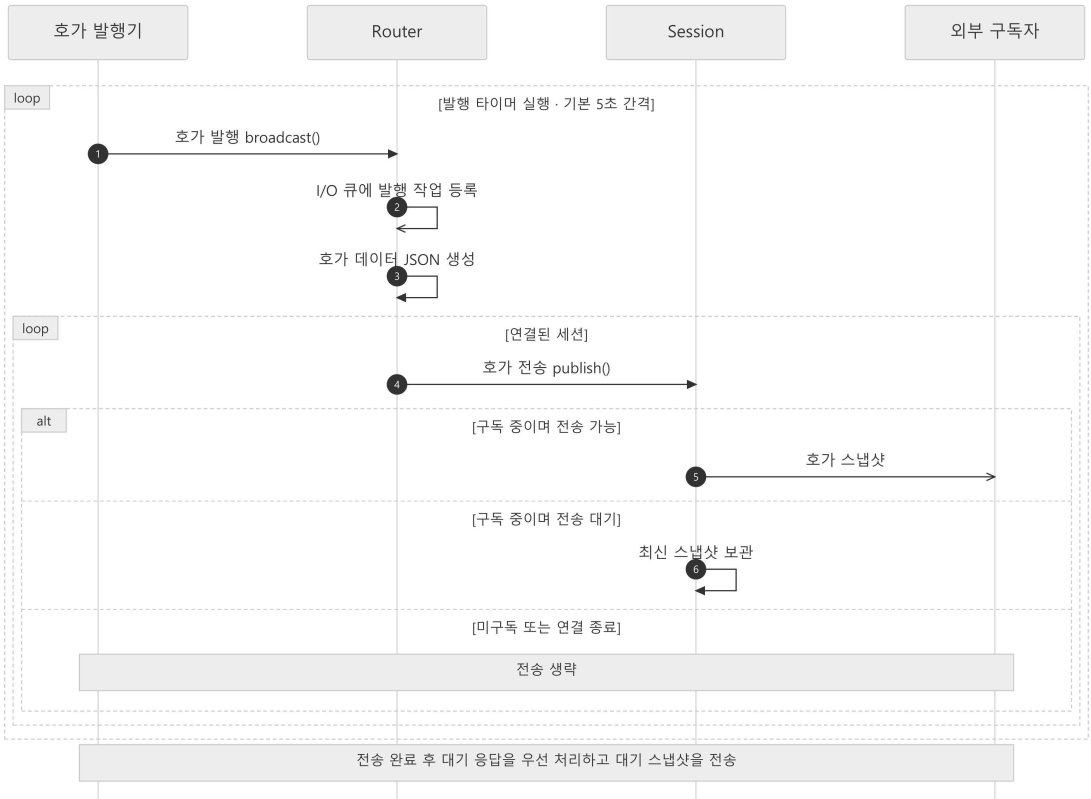


[그림 11-8] 호가 스냅샷 생성 및 갱신 시퀀스

매수호가는 가격 내림차순, 매도호가는 오름차순으로 구성한다. 실행 중 스냅샷 갱신은 주문 처리와 같은 Worker에서 수행하며, 외부 전송은 다음 절의 발행 타이머를 통해 이루어진다.

11-9. 호가 데이터 발행

Publisher가 기본 5초 간격으로 캐시된 호가 스냅샷을 구독 세션에 전달하는 순서이다. 엔진 Worker에서 발행 작업을 I/O 큐에 등록하고, I/O 스레드에서 JSON 변환과 세션별 WebSocket 전송을 수행한다.



[그림 11-9] 호가 데이터 발행 시퀀스

다른 메시지를 전송 중인 세션에는 최신 스냅샷 하나를 보관한다. 전송 완료 후 대기 중인 주문 응답을 우선 처리하고 스냅샷을 전송한다. 구독하지 않았거나 종료된 연결은 발행 대상에서 제외한다.

12. 알고리즘 구성 (Algorithm)

12-1. 가격·시간 우선 매칭 알고리즘 (C++ 체결 엔진)

자료구조 : 종목별 매수·매도 호가창을 가격 레벨 단위로 관리하고, 레벨 내부는 도착 순서대로 FIFO 큐로 구성한다. 주문 ID로 주문을 즉시 찾기 위한 해시 테이블을 함께 둔다.

① 신규 주문이 도착하면 반대편 호가창의 최우선 호가(매수 주문이면 가장 낮은 매도 호가)와 가격을 비교한다.

② 지정가 주문은 최우선 호가가 지정가 이내일 때, 시장가 주문은 가격과 관계없이 체결을 진행한다.

③ 같은 가격 레벨에서는 큐의 앞쪽, 즉 먼저 들어온 주문부터 체결한다(시간 우선).

④ 레벨의 수량을 모두 소진하면 다음 우선순위 가격 레벨로 넘어간다(가격 우선).

⑤ 지정가 주문의 남은 수량은 자신의 호가창 해당 가격 레벨 큐의 뒤에 추가한다.

⑥ [취소] 해시 테이블로 주문을 찾아 큐에서 제거한다. [정정] 수량 감소는 그대로 반영하고, 가격 정정은 기존 주문을 취소한 뒤 새 주문으로 처리하여 시간 우선순위를 새로 부여한다.

⑦ 모든 주문·체결을 순서대로 로그에 남기며, 장애 후 로그를 재생하여 호가창을 복원한다.

<예시> 매수 지정가 10,260원 × 250주 주문 도착

매도 호가	대기 주문 (도착 순)	체결 전	체결 후
10,260	S3 200주	200주	S3 잔량 100주
10,250 (최우선)	S1 100주 → S2 50주	150주	모두 체결

S1 100주와 S2 50주가 10,250원에, S3 100주가 10,260원에 체결되어 매수 주문 250주가 모두 체결된다. 현재가는 10,260원이 되고, 매도 호가창에는 S3의 잔량 100주가 남는다.

12-2. 봇 확률 기반 주문 생성 알고리즘

봇은 종목마다 아래 세 파라미터를 받아 동작하며, 이 값은 2시간 구간마다 새로 정해진다.

파라미터	의미	예시
기준 가격 (P50)	매수(매도) 확률이 정확히 50%가 되는 가격	10,000원
확률 변동 폭 (step)	확률이 1%p 바뀌는 데 필요한 가격 변화	20원
분당 참여자 수 (N)	페르소나의 시장 참여 빈도. 클수록 봇 호출 주기가 짧아진다.	분당 3회

① 매수 확률 : 매수 확률(%) = 50 + (P50 − 현재가) ÷ step 이며 0~100% 범위로 제한한다. 매도 봇은 부호를 반대로 적용한다. 예를 들어 P50 = 10,000원, step = 20원이면 현재가 9,800원에서 매수 확률 60%, 10,200원에서 40%

이다. 같은 페르소나의 매수 기준 가격과 매도 기준 가격의 차이가 그 페르소나의 시장 심리를 나타낸다.

② 호출 주기 : 평균 60 ÷ N초 간격으로 호출되며, 간격은 지수 분포를 따르는 무작위 값으로 정해 모든 봇이 같은 박자로 주문하지 않게 한다.

③ 주문 수량 : 확률로 주문 여부를 정한 뒤 같은 확률로 시행을 반복해 실패할 때까지 성공한 횟수를 수량으로 한다. 평균 수량은 1 ÷ (1 − p)이며, 반복 확률과 주문당 수량에 상한을 둔다.

④ 주문 가격 : 확률식을 역산한 의향 가격 P50 + (50 − u) × step(u는 0~100 균등 난수)을 호가 단위에 맞춘 지정가로 낸다. 봇마다 다른 가격에 주문이 쌓여 호가창이 형성된다. 프로토타입 검증에서 극단적인 의향 가격이 급락을 일으켜, 봇 주문 가격은 현재가 ± 페르소나별 범위 안으로 제한한다(15-1 참고).

12-3. LLM 파라미터 검증·반영 알고리즘

① 형식 검사: 정해진 JSON 형식(페르소나 × 매수/매도 × 종목별 P50, step, N)으로 파싱되는지 확인한다.

② 값 범위 보정: 기준 가격은 현재가 대비 허용 범위 안으로, step과 N은 양수이며 상한 이하로 보정한다.

③ 변경 폭 제한: 이전 구간 값 대비 1회 변경 폭을 넘으면 제한값으로 맞춘다. 이상값 항목은 이전 값을 유지한다.

④ 점진 반영: 각 페르소나의 여러 봇 인스턴스 중 신규 반영 비율만큼만 새 파라미터를 적용한다. 비율은 시장 사실성 검증 결과[6][7]를 보며 조정한다.

⑤ 기록: 입력·출력·검증 결과·적용 여부를 모두 저장하여 시장 변화를 추적·재현할 수 있게 한다.

12-4. 기간 수익률 및 순위 집계 알고리즘

대회 종류	산정 구간 (2027년 1월 참가 예시)	순위 기준
1개월 대회	2027.01.01 ~ 2027.01.31 (해당 연도 첫 번째 1개월)	구간 내 수익률
3개월 대회	2027.01.01 ~ 2027.03.31 (해당 연도 첫 번째 3개월)	구간 내 수익률
6개월 대회	2027.01.01 ~ 2027.06.30 (해당 연도 첫 번째 6개월)	구간 내 수익률
1년 대회	2027.01.01 ~ 2027.12.31 (해당 연도 첫 번째 1년)	구간 내 수익률

● 기간 수익률 = (구간 종료 시점 평가자산 − 구간 시작 시점 평가자산) ÷ 구간 시작 시점 평가자산. 시드머니 외 추가 입금은 없는 것을 기본으로 한다.

● 매 장 마감 시 계정별 평가자산을 스냅샷으로 저장하고, 진행 중인 구간의 순위는 Redis Sorted Set으로 갱신하여 빠르게 조회한다. Sorted Set은 점수 순 정렬을 유지하며 순위 조회와 갱신이 O(log N)이고, 동점은 멤버 사전순으로

구분된다[10].

- 구간 도중 참가: 참가 시점의 평가자산을 해당 구간의 기준 자산으로 삼아 진행 중인 구간에 바로 편입하고, 구간 종료 시점까지의 수익률로 순위를 매긴다.
- 구간이 끝나면 순위를 확정·보관하고, 다음 구간은 자동으로 시작된다.

13. 데이터베이스 테이블 구조 (DB Table)

13-1. 주요 데이터와 저장 방식

데이터	내용	저장 방식
종목·시장	종목 정보, 섹터, 상장 정보, 시장 설정	MySQL
주문·체결	주문 내역, 체결 내역(대용량, 지속 증가)	엔진 로그 + MySQL 배치 저장
시세	현재가, 호가 스냅샷, 분봉·일봉 집계	Redis(실시간) + MySQL(이력)
계좌·포지션	예수금, 보유 종목, 일별 평가자산 스냅샷	MySQL
대회·랭킹	기간 구간 정보, 구간별 수익률·순위	MySQL + Redis Sorted Set
봇 파라미터	페르소나·봇 정의, 2시간 구간별 파라미터 및 변경 이력	MySQL
사건	사건 일정, 유형, 영향 파라미터	MySQL
AI 기록	LLM 입력·출력, 검증 결과, 적용 여부, 생성 콘텐츠	MySQL

13-2. 테이블 명세

10장의 도메인 클래스를 기준으로 한 MySQL 테이블 구성이다. PK는 기본키, FK는 외래키를 뜻한다.

테이블	주요 컬럼	설명
member	id(PK), nickname(UNIQUE), password_hash, joined_at, deleted_at	회원
account	id(PK), member_id(FK), cash, reserved_cash	계좌 예수금과 묶어 둔 금액
holding	id(PK), owner_type, owner_id, stock_id(FK), quantity, reserved_quantity, avg_price	회원·봇 보유 종목
stock	id(PK), name, sector, base_price, tick_size, listed_at	가상 종목
orders	id(PK), owner_type, owner_id, stock_id(FK), side, order_type, price, quantity, filled_quantity, status, created_at	회원·봇 공용 주문
execution	id(PK), buy_order_id(FK), sell_order_id(FK), stock_id(FK), price, quantity, executed_at	체결 내역
candle	stock_id(FK), interval_type, start_time, open, high, low, close, volume	분봉·일봉 집계
asset_snapshot	id(PK), account_id(FK), snapshot_date, total_asset	장 마감 평가자산
ranking_result	id(PK), period_type, period_start, member_id(FK), base_asset, return_rate, rank, joined_at	확정된 구간별 순위
persona	id(PK), name, investor_type, traits	페르소나 정의(유형, 성향 항목)
bot	id(PK), persona_id(FK), side, new_param_applied	봇 인스턴스
bot_parameter	id(PK), persona_id(FK), stock_id(FK), side, p50, step, n, interval_start, llm_log_id(FK)	구간별 파라미터 이력
event	id(PK), scope, stock_id, sector, event_type, content, occurred_at	거시 환경·개별 사건
event_impact	id(PK), event_id(FK), persona_id(FK), size, delay_minutes, duration_minutes	페르소나별 사건 충격
llm_log	id(PK), call_type, input, output, validation_result, applied, called_at	LLM 입력·출력·적용 기록

13-3. Redis 키 구성

키	자료형	내용
price:{stockId}	Hash	현재가, 등락률, 거래량
orderbook:{stockId}	Hash	호가 스냅샷
ranking:{periodType}:{periodStart}	Sorted Set	진행 중인 구간의 회원별 수익률 순위

14. 시나리오 (Scenario)

[회원가입]

1. 참가자 A가 2027년 1월 9일 회원가입을 한다. 닉네임 중복 확인을 거쳐 계좌가 생성되고 시드머니가 지급된다. A는 진행 중인 1개월·3개월·6개월·1년 구간에 가입 시점의 평가자산을 기준으로 자동 편입된다.

[시세 확인]

2. A는 메인 페이지에서 반도체 섹터가 오르는 것을 확인하고 종목 "가상전자"의 종목 페이지에 들어간다. 이때 해당 종목의 시세·호가 구독이 시작된다.

[주문과 부분 체결]

3. A는 지정가 10,200원에 20주 매수 주문을 낸다. 서버는 가격제한폭·호가 단위·주문 가능 금액을 확인하고 주문 금액을 묶어 둔 뒤 체결 엔진에 전달한다.
4. 10,200원에 대기 중인 매도 주문이 5주뿐이어서 5주만 체결되고, 남은 15주는 매수 호가창에서 대기한다. A에게 체결 알림이 전송된다.

[사건 발생과 봇 반응]

5. 10:20에 "가상전자 분기 실적, 시장 기대치 상회" 사건이 발생한다. LLM이 페르소나별 충격 크기를 산출하고 이를 기준 가격 이동으로 즉시 반영한다.
6. 개인 단기 투자형 봇은 즉시·짧게, 기관 액티브 펀드형 봇은 늦게·길게 반응한다. 매수 확률이 오른 봇들의 주문이 늘어 가격이 오르고, A의 남은 15주도 체결된다.

[주문 정정]

7. A는 보유한 20주 중 10주를 지정가 10,600원에 매도 주문했다가, 상승세가 둔화되자 내 자산 페이지에서 가격을 10,550원으로 정정한다.

[구간 전환]

8. 11:00 직전 스케줄러가 LLM을 호출한다. 직전 구간 시세와 실적 사건이 입력되어 페르소나별 새 파라미터가 산출되고, 검증을 통과한 값이 신규 반영 비율만큼 봇에 적용된다.

[장 마감과 순위 확인]

9. 15:30 폐장 후 A의 평가자산 스냅샷이 저장되고 기간 수익률과 Redis 순위가 갱신된다.
10. A는 대회 페이지에서 1개월 구간의 자신의 순위와 수익률을 확인한다. 랭킹에는 A의 참가 시작일(01.09)이 함께 표시된다.

[구간 종료]

11. 1월 31일 장 마감 시 1개월 구간의 순위가 확정·보관되고, 2월 1개월 구간이 자동으로 시작된다. A는 별도 신청 없이 다음 구간에 참여한다.

15. 역할 분담 및 개발일정

15-1. 개발 진행 현황

- 개발 환경: Spring Boot·Java 21·Gradle 기반 서버 프로젝트와 Docker Compose로 실행하는 MySQL 8 로컬 환경을 구성하고, 접속 정보는 환경 변수로 분리했다.
- 협업 규칙: GitHub 저장소를 main(발표용 안정 버전)과 develop(개발 통합) 브랜치로 운영한다. 네이밍·커밋·PR 컨벤션 문서와 이슈·PR 템플릿을 두고, 팀원 1인 이상 승인 후 머지한다.

- 도메인 엔티티: 회원, 봇, 봇 파라미터 이력, 종목, 주문(회원·봇 공용), 보유 주식 엔티티와 저장소 계층을 구현하고 테스트를 마쳤다.
- 프로토타입 검증: 핵심 기능인 봇 구동과 회원 매수·매도를 별도 브랜치에서 프로토타입으로 먼저 검증했다. 종목 1개, 페르소나 3종(공격형·안정형·추세형)의 매수·매도 봇으로 단순화하고, LLM 대신 고정 파라미터를 사용했다.
  - 체결 엔진 인터페이스 뒤에 Java 인메모리 매칭을 두어 이후 C++ 엔진으로 교체할 수 있게 했다.
  - 회원 매수·매도와 체결 반영, 봇 확률 기반 주문, 봇 스케줄러와 장 운영(개장·폐장)을 구현하고 자동화 테스트 168개를 통과했다.
  - 주문 시 금액·수량 묶기, 주문 저장 후 엔진 제출, 비관적 락을 통한 동시 수정 방지, 개장 시 남은 미체결 주문 정리를 적용했다.
- 프로토타입 검증 결과
  - 첫 실행에서 매도 봇이 극단적인 의향 가격을 그대로 내어 가격이 한 번에 급락하고 거래가 끊겼다. 봇 주문 가격을 현재가 ± 페르소나별 범위(공격형 60원, 추세형 40원, 안정형 20원)로 제한하여 해결했다.
  - 9분 실행에서 같은 페르소나 봇끼리 체결되는 자기 거래 비율이 51.6%로 높게 나타났다. 기준 가격 간격 조정이나 같은 페르소나 간 체결 제한을 검토한다.
  - 남은 과제: 봇이 마지막 체결가 대신 호가창을 보고 판단하는 방식, 추세 추종형이 실제 추세를 따르게 하는 방식, 30분 이상 장시간 실행 재검증, 실제 LLM 파라미터 산출 연동
- 11월 구현 현황 (최종 설계제안서 제출 시점)
  - 체결 엔진: 지정가 매칭, 주문 전체 취소, 체결 시각 응답을 구현했고, 매수·매도 5단계 호가 구독·발행은 검토 중이다.
  - 서버: 종목 차트 조회 API와 실시간 캔들 갱신, 체결 내역(trades) 저장, 봇끼리의 체결을 주기적으로 정리하는 스케줄러를 구현했다. 주문·취소·현재가 API와 세션 인증은 검토 중이다.
  - AI: 구간별 봇 파라미터 산출 모듈(LLM 프롬프트 생성, 응답 검증·보정)과 사건 모듈(1년치 사건 일정, 페르소나별 기준 가격 변화 계산)을 Python으로 구현했다. LLM API 연동은 M3에서 진행한다.

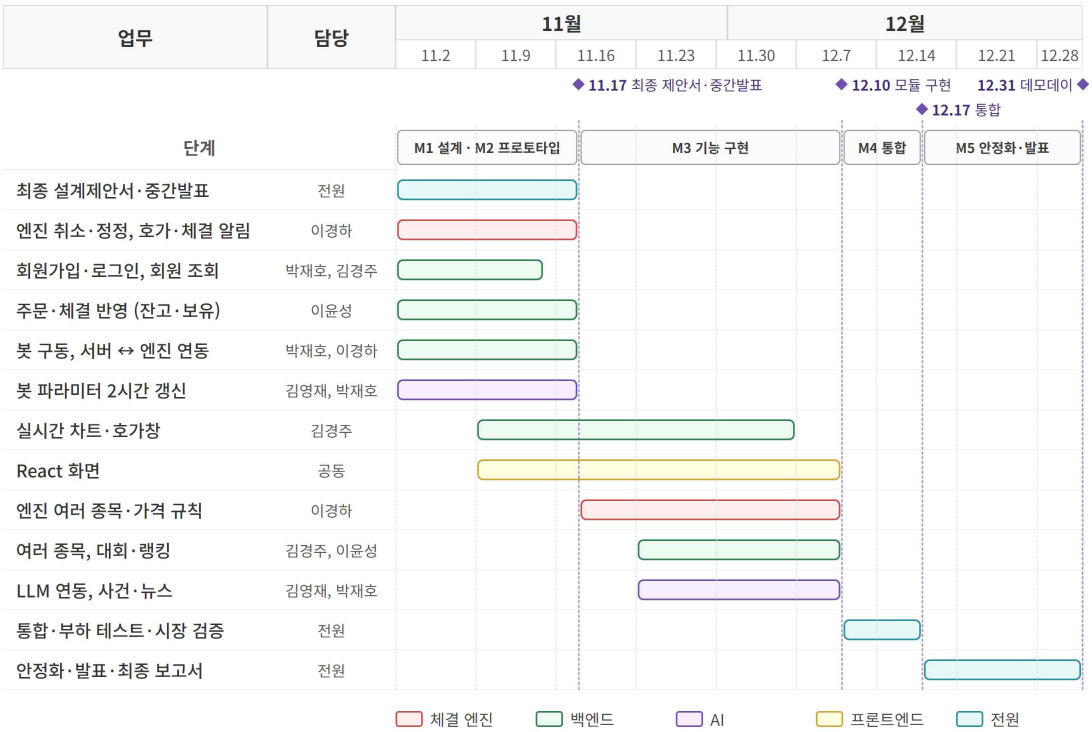
15-2. 역할 분담

이름	파트	담당 업무
김영재 (팀장)	AI · 프로젝트 관리	LLM 기반 봇 파라미터 산출(2시간 구간), 사건·뉴스 생성, 시장 사실성 검증 / 일정·문서 관리(설계 제안서, 회의록)
박재호	백엔드 · 봇	AI 투자자 봇(확률 기반 주문 생성, 호출 스케줄링), 회원가입·로그인, DB 설계(엔티티·저장소)
이윤성	백엔드 · 주문	주문 접수·취소·정정, 현재가 조회, 체결 결과 반영(잔고·보유 수량), 장애 복구
김경주	백엔드 · 실시간	WebSocket(STOMP) 실시간 전송, 차트(캔들)·호가창 API, 회원 정보 조회, 협업 규칙(브랜치·커밋·PR)
이경하	체결 엔진	종목별 호가창과 가격·시간 우선 매칭, 주문 취소·정정, 실시간 호가·체결 알림 전송, 여러 종목·가격 제한폭·호가 단위, 서버 연동 명세
공동	프론트엔드	React 화면: 각자 맡은 기능의 화면을 함께 구현

15-3. 개발 일정 (마일스톤)

단계	기간	목표	산출물
M1 설계 완료	~ 11.17	정보 모델링, 유스케이스, 시퀀스 다이어그램, ERD, 역할·일정, 컨벤션, 기술 스택 확정	최종 설계제안서
M2 프로토타입	10.26 ~ 11.17	종목 1개·봇 6개로 시장이 움직이는지 검증: 회원가입·로그인, 매수·매도·취소, 현재가, 엔진 연동, 봇 파라미터 2시간 갱신	프로토타입 시연, 중간발표(11.17)
M3 기능 구현	11.18 ~ 12.10	여러 종목, 가격제한폭·호가 단위, 실시간 차트·호가창, 계좌·보유 종목, 기간별 대회·랭킹, 페르소나 확장, LLM 연동, 사건·뉴스, React 전체 화면	소스코드, 주간보고서
M4 시스템 통합	12.11 ~ 12.17	엔진·서버·봇·AI·화면 전체 연동, 부하 테스트, 시장 사실성 검증	통합 테스트 결과
M5 안정화·발표	12.18 ~ 12.31	오류 수정, 장애 복구 점검, 발표·시연 준비, 데모데이(12.31)	최종 발표 자료, 시연 영상, 최종 설계 보고서

15-4. 주차별 일정표



[그림 15-1] 주차별 일정표 (11.17 최종 제안서·중간발표, 12.10 모듈 구현, 12.17 통합, 12.31 데모데이)

15-5. 위험 요소 및 대응

위험 요소	대응
LLM 응답 오류·이상값	형식 검사, 값 범위 보정, 실패 시 이전 파라미터 유지, 입출력 기록 보관
LLM 응답 지연·장애	구간 시작 전에 미리 호출하고, 시간 초과 시 이전 파라미터로 다음 구간을 진행
가격 폭주·붕괴	일일 가격제한폭(±30%)[2], 봇 주문 가격의 현재가 기준 범위 제한, 1회 파라미터 변경 폭 제한, 신규 반영 비율 조절
엔진 장애	주문·체결 로그 재생으로 호가창 복구
봇 주문 폭증	주문 속도 제한, 종목 단위 처리 분리, 부하 테스트
유동성 부족	마켓메이커형·노이즈 트레이더 봇으로 호가 유지
비현실적인 가격 움직임	통계적 특징 측정[6][7]으로 사건 확률과 파라미터 조정
봇 간 자기 거래 과다	자기 거래 비율 측정, 기준 가격 간격 조정 또는 같은 페르소나 간 체결 제한
중도 참가자 형평성	구간별 참가 시작일을 랭킹에 함께 표시

16. 관련 기술 및 개념

- 가격·시간 우선 호가 매칭

한국거래소 정규장은 연속매매 방식이며 가격 우선·시간 우선 원칙으로 체결한다[2]. 지정가 주문 장부의 구조와 통계적 특성은 학술적으로 정리되어 있으며[5], 본 프로

젝트의 체결 엔진은 이를 참고하여 지정가·시장가 주문을 처리한다.

- 에이전트 기반 시장 시뮬레이션

여러 유형의 거래 에이전트가 각자의 규칙으로 주문을 내고, 그 상호작용으로 가격이 형성되는 시뮬레이션 방식이다. ABIDES는 이산 사건 시뮬레이터 위에서 여러 거래 에이전트가 거래소 에이전트를 통해 거래하는 구조를 제시한다[4]. 본 프로젝트의 AI 투자자 봇은 이 접근을 따른다.

- 체결 엔진 연동 방식

C++ 체결 엔진은 별도 프로세스로 실행하고, Spring 서버와 WebSocket으로 JSON 메시지를 주고받는다. 프로세스가 분리되어 있어 엔진과 서버를 따로 개발하고 재시작할 수 있다. JNI[9]나 JDK 22에서 정식 기능이 된 FFM API[8]처럼 같은 프로세스 안에서 네이티브 코드를 호출하는 방식은 호출 지연이 작은 대신 결합도가 높아, 부하 시험에서 지연이 문제가 될 경우 대안으로 검토한다.

- LLM 기반 파라미터 산출

한국거래소는 개인·기관·외국인 등 투자자 유형별 거래실적을 공개한다[3]. 본 프로젝트는 이 통계와 직전 구간의 시세·사건, 페르소나 정의를 LLM에 입력으로 전달하고, 페르소나별 주문 파라미터를 정해진 JSON 형식으로 받는다. LLM 출력은 같은 입력에도 달라질 수 있으므로 형식·범위·변경 폭을 검증한 뒤 반영하고, 실패하면 이전 값을 유지한다.

- LLM 기반 트레이딩 에이전트

TradingAgents는 분석·토론·위험 관리·거래 역할을 맡은 여러 LLM 에이전트가 협업해 매매를 결정하는 구조를 제시했다[13]. 판단 과정을 문장으로 설명할 수 있지만 판단마다 모델을 호출해야 해 실시간 시장에는 느리고 비용이 크며, 같은 입력에도 결과가 달라질 수 있다. 본 프로젝트는 개별 매매 판단을 봇의 확률 규칙에 맡기고, LLM은 2시간 구간 단위의 파라미터 산출과 콘텐츠 생성에 사용한다.

- JWT 인증

JWT는 두 당사자 사이에 전달할 정보를 JSON으로 표현하고 서명할 수 있는 간결한 토큰 형식이다[14]. 서버가 로그인 상태를 별도로 저장하지 않아도 요청마다 토큰을 검증할 수 있어, REST API와 WebSocket 연결을 같은 방식으로 인증할 수 있다.

- 시장 사실성 검증

실제 주가 수익률의 통계적 특징(stylized facts)은 Cont(2001)가 정리했다[6]. 후속

연구는 Cont의 11개 사실 중 수익률의 자기상관 부재, 두꺼운 꼬리, 변동성 군집, 절대 수익률 자기상관의 느린 감소 등을 최근 주식시장 데이터에서 다시 확인했다[7]. 가상 시장이 이러한 특징을 재현하는지 측정하고, 그 결과로 사건 확률과 파라미터를 조정한다.

- Redis Sorted Set

Redis의 Sorted Set은 점수 순 정렬을 유지하며, 순위 조회와 갱신이 O(log N)이고 동점은 멤버의 사전순으로 구분된다[10]. 본 프로젝트는 진행 중인 기간 대회의 수익률 순위를 이 구조로 집계한다.

- WebSocket과 STOMP

WebSocket은 하나의 TCP 연결에서 양방향 통신을 제공하며, 저지연·고빈도·대용량 메시지가 필요한 금융 서비스처럼 실시간성이 높은 환경에 적합하다. Spring은 WebSocket 위에서 STOMP 하위 프로토콜을 지원한다[11].

- React

React는 컴포넌트 단위로 사용자 인터페이스를 구성하는 라이브러리이다[12]. 본 프로젝트는 시세 차트, 호가창, 주문 입력, 랭킹 표를 컴포넌트로 구성하고, WebSocket 구독으로 받은 데이터를 화면에 반영한다.

17. 기대효과

- 가격 형성 과정의 체험

시장 상황과 투자자 유형별 특성을 반영해 움직이는 AI 투자자와 함께 거래하며, 사용자의 주문이 가격에 영향을 주고 같은 사건에도 투자자 유형별 반응이 달라지는 환경에서 수요·공급에 따른 가격 형성과 기대치 대비 결과를 고민하며 투자 판단을 연습할 수 있다.

- 상시 참여 가능한 기간별 대회

1개월·3개월·6개월·1년 단위의 논리적 대회에 자동 편입되므로, 원하는 때에 참여해 같은 기간의 다른 참가자와 수익률로 실력을 겨룰 수 있다.

- 재현 가능한 시장 환경

가상 기업과 가상 사건을 사용하므로 다양한 시장 상황을 의도적으로 연출할 수 있고, 모든 LLM 입력·출력과 파라미터 변경 이력이 기록되어 시장 변화를 추적하고 재현할 수 있다.

- 확장성

시장 단위로 엔진과 봇을 분리하여 이후 여러 시장으로 확장할 수 있다.

18. 개발도구 및 개발환경

구분	내용
언어	Java 21, C++, JavaScript
프레임워크·라이브러리	Spring Boot(REST API, WebSocket/STOMP), React, Boost.Asio·Boost.Beast
빌드 도구	Gradle, CMake
데이터베이스	MySQL 8, Redis
실행 환경	Docker Compose 기반 로컬 환경 (접속 정보는 환경 변수로 분리)
외부 서비스	외부 LLM API
형상 관리·협업	GitHub (main·develop 브랜치, 이슈·PR 템플릿, 리뷰 후 머지)

19. 참고문헌

※ URL은 2026.10.12 기준으로 접속하여 확인한 자료이다. 단, [14]는 2026.10.27 기준이다.

[1] 한국투자증권, 모의투자안내 - 주식/선물옵션 모의투자 (증권사 모의투자 서비스 방식)

https://securities.koreainvestment.com/main/research/virtual/_static/TF07da010000.jsp

[2] 한국거래소(KRX), Guide to Trading in the Korean Stock Market (정규장 연속매매의 가격·시간 우선 원칙, 주문 유형, 일일 가격제한폭 기준가격 대비 ±30%, 매매 시간 09:00~15:30)

https://global.krx.co.kr/contents/GLB/01/0109/0109000000/guide_to_trading_in_the_korean_stock_market.pdf

KRX Global, Trading Hours

https://global.krx.co.kr/contents/GLB/06/0602/0602020204/GLB0602020204T1.jsp

[3] 한국거래소, KRX 정보데이터시스템(KRX Data Marketplace) 투자자별 거래실적 (개인·기관·외국인 투자자 유형별 거래 통계)

https://data.krx.co.kr/contents/MDC/MDI/outerLoader/index.cmd?screenId=MDCSTAT022&locale=ko_KR&kosdaqGlobalYn=1

[4] D. Byrd, M. Hybinette, T. H. Balch, "ABIDES: Towards High-Fidelity Multi-Agent Market Simulation", Proceedings of the 2020 ACM SIGSIM Conference on Principles of Advanced Discrete Simulation (SIGSIM-PADS '20), pp. 11-22, 2020. (arXiv:1904.12066) (이산 사건 기반 다중 에이전트 시장 시뮬레이션 구조)

https://arxiv.org/abs/1904.12066

[5] M. D. Gould, M. A. Porter, S. Williams, M. McDonald, D. J. Fenn, S. D. Howison, "Limit Order Books", Quantitative Finance, 2013. (arXiv:1012.0349) (지정가 주문 장부의 구조·통계적 특성·모형 조사)

https://arxiv.org/abs/1012.0349

[6] R. Cont, "Empirical properties of asset returns: stylized facts and statistical issues", Quantitative Finance, 1(2), 223-236, 2001. DOI: 10.1080/713665670 (실제

자산 수익률의 통계적 특징)
https://ideas.repec.org/a/taf/quantf/v1y2001i2p223-236.html
[7] E. Ratliff-Crain, C. M. Van Oort, J. Bagrow, M. T. K. Koehler, B. F. Tivnan, "Revisiting Cont's Stylized Facts for Modern Stock Markets", arXiv:2311.07738 (Cont의 11개 사실 중 8개가 최근 데이터에서 확인됨)
https://arxiv.org/abs/2311.07738
[8] OpenJDK, JEP 454: Foreign Function & Memory API (JDK 22 기능)
https://openjdk.org/jeps/454
OpenJDK, JDK 22 (2024.03.19 GA)
https://openjdk.org/projects/jdk/22/
[9] Oracle, Java Native Interface Specification (Java SE 21)
https://docs.oracle.com/en/java/javase/21/docs/specs/jni/design.html
[10] Redis, Redis sorted sets (랭킹 용도, ZADD/ZRANK/ZREVRANK의 O(log N) 복잡도, 동점 시 사전순)
https://redis.io/docs/latest/develop/data-types/sorted-sets/
[11] Spring Framework Reference, WebSockets (WebSocket과 STOMP 지원, 저지연·고빈도·대용량 환경에 적합)
https://docs.spring.io/spring-framework/reference/web/websocket.html
[12] React, The library for web and native user interfaces
https://react.dev/
[13] Y. Xiao, E. Sun, D. Luo, W. Wang, "TradingAgents: Multi-Agents LLM Financial Trading Framework", arXiv:2412.20138, 2024. (LLM 다중 에이전트 기반 매매 프레임워크)
https://arxiv.org/abs/2412.20138
[14] M. Jones, J. Bradley, N. Sakimura, "JSON Web Token (JWT)", RFC 7519, IETF, 2015. (JSON 기반 토큰 형식과 서명 방식)
https://www.rfc-editor.org/rfc/rfc7519